

6-1장 정리

▼ 6.1 이미지 분류를 위한 신경망

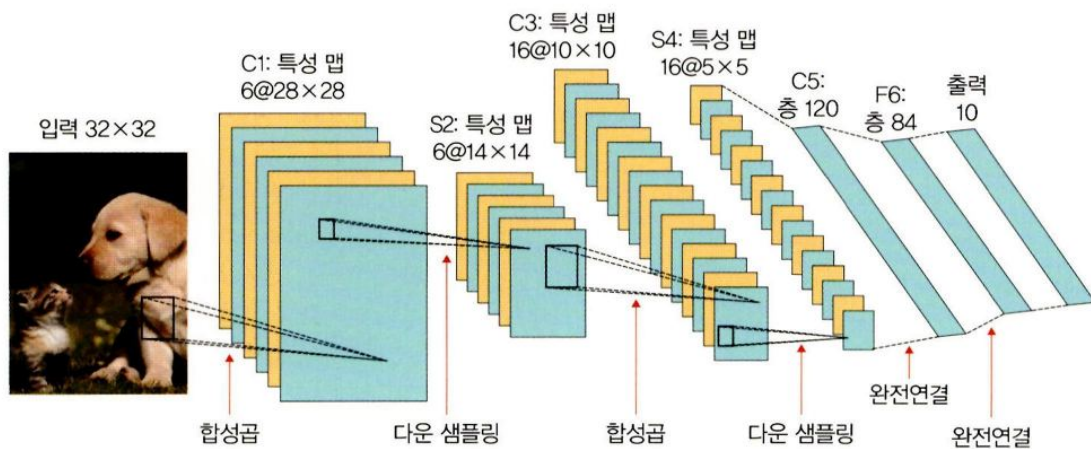
입력 데이터로 이미지를 사용한 분류(classification)는 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것임. 이미지 분류(image classification)에서 주로 사용되는 합성곱 신경망의 유형을 알아보겠습니다.

6.1.1 LeNet-5

LeNet-5는 합성곱 신경망이라는 개념을 최초로 안 르쿤(Yann LeCun)이 개발한 구조임. 1995년 안 르쿤, 레옹 보토(Leon Bottu), 요슈아 벤지오(Yosua Bengio), 패트릭 하프너(Patrick Haffner)가 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조 LeNet-5를 발표했는데, 그것이 현재 CNN의 초석이 되었음. LeNet-5는 합성곱(convolutional)과 다운 샘플링(sub-sampling)(혹은 풀링)을 반복적으로 거치면서 마지막에 완전연결층에서 분류를 수행함.

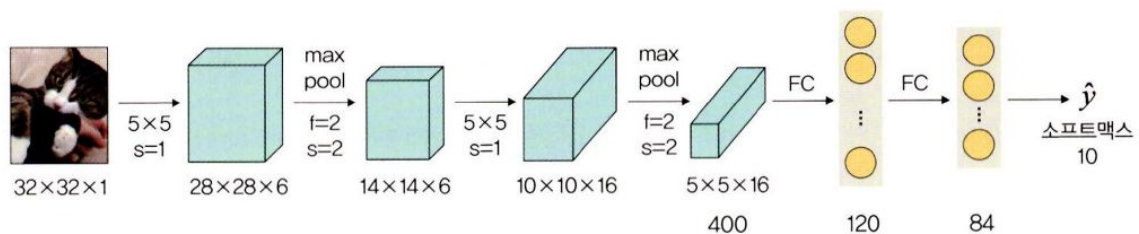
다음 그림을 이용하여 구체적으로 살펴보면 C1에서 5×5 합성곱 연산 후 28×28 크기의 특성 맵(feature map) 여섯 개를 생성함. S2에서 다운 샘플링하여 특성 맵 크기를 14×14 로 줄임. 다시 C3에서 5×5 합성곱 연산하여 10×10 크기의 특성 맵 16개를 생성하고, S4에서 다운 샘플링하여 특성 맵 크기를 5×5 로 줄임. C5에서 5×5 합성곱 연산하여 1×1 크기의 특성 맵 120개를 생성하고, 마지막으로 F6에서 완전연결층으로 C5의 결과를 유닛(unit)(또는 노드) 84개에 연결시킴. 이때 C로 시작하는 것은 합성곱층을 의미하고, S로 시작하는 것은 풀링층을 의미함. 또한, F로 시작하는 것은 완전연결층을 의미함.

▼ 그림 6-1 LeNet-5



LeNet-5를 사용하는 예제를 구현해 봄. 앞 장에서 사용한 개와 고양이 데이터셋을 다시 사용함. 우리가 구현할 신경망은 다음 그림과 같음.

▼ 그림 6-2 LeNet-5 예제 신경망



32×32 크기의 이미지에 합성곱층과 최대 풀링층이 쌍으로 두 번 적용된 후 완전연결층을 거쳐 이미지가 분류 되는 신경망임.

신경망에 대한 자세한 설명은 다음 표와 같음.

표 6-1 LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	–	–	–
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	–
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	–
완전연결층	–	120	–	–	렐루(ReLU)
완전연결층	–	84	–	–	렐루(ReLU)
완전연결층	–	2	–	–	소프트맥스 (softmax)

예제를 진행하기 위해 아나콘다 프롬프트에서 tqdm 라이브러리를 설치함. 라이브러리는 다음과 같이 두 가지 방법으로 설치할 수 있음.

```
> pip install --user tqdm
```

혹은

```
> conda install -c conda-forge tqdm
```

‘tqdm’은 아랍어로 progress(진행 상태)라고도 함. 즉, 진행 상태를 바(bar) 형태로 가시화하여 보여 줌. 주로 모델 훈련에 대한 진행 상태를 확인하고자 할 때 사용함.

설치가 완료되었다면 필요한 라이브러리를 호출함.

코드 6-1 필요한 라이브러리 호출

```
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms ----- 이미지 변환(전처리) 기능을 제공하는 라이브러리
from torch.autograd import Variable
from torch import optim ----- 경사 하강법을 이용하여 가중치를 구하기 위한 옵티마이저 라이브러리
import torch.nn as nn
import torch.nn.functional as F
import os ----- 파일 경로에 대한 함수들을 제공
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm ----- 진행 상황을 가시적으로 표현해 주는데, 특히 모델의 학습 결과를
import random ----- 확인하고 싶을 때 사용하는 라이브러리
from matplotlib import pyplot as plt

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu") -----
                                     파이토치는 텐서플로와 다르게 GPU를 자동으로 할당해 주지 않기 때문에 GPU 할당을
                                     모델과 데이터에 선언해 주어야 합니다. 단 이 장에서는 CPU를 사용합니다.
```

먼저 모델 학습에 필요한 데이터셋의 전처리(예 텐서 변환)가 필요함.

코드 6-2 이미지 데이터셋 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5,1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]), ----- ①
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase): ----- ②
        return self.data_transform[phase](img)
```

① 토치비전(torchvision) 라이브러리를 이용하면 이미지에 대한 전처리를 손쉽게 할 수 있음.

`torchvision.transforms` 에서 사용하는 파라미터는 다음과 같음. 이미 5장에서 살펴보았지만 다시 한 번 정리해보겠음.

```
train_transforms = transforms.Compose(  
    (a)  
    [transforms.RandomResizedCrop(resize, scale=(0.5,1.0)),  
     (b)  
     transforms.RandomHorizontalFlip(),  
     (c)  
     transforms.ToTensor(), transforms.Normalize(mean, std)])  
    (d)                (e)
```

(a) `transforms.Compose` : 이미지를 변형할 수 있는 방식들의 묶음임.

(b) `transforms.RandomResizedCrop` : 입력 이미지를 주어진 크기(resize: 224×224)로 조정함. 또한, scale은 원래 이미지를 임의의 크기(0.5~1.0(50~100%))만큼 면적을 무작위로 자르겠다는 의미임.

(c) `transforms.RandomHorizontalFlip` : 주어진 확률로 이미지를 수평 반전시킴. 이때 확률 값을 지정하지 않았으므로 기본값인 0.5의 확률로 이미지들이 수평 반전됨. 즉, 훈련 이미지 중 반은 위아래 뒤집힌 상태로 두고, 반은 그대로 사용함.

(d) `transforms.ToTensor` : ImageFolder 메서드를 비롯해서 torchvision 메서드는 이미지를 읽을 때 파이썬 이미지 라이브러리인 PIL을 사용함. PIL을 사용해서 이미지를 읽으면 생성되는 이미지는 범위가 [0, 255]이며, 배열의 차원이 (높이 H×너비 W×채널 수 C)로 표현됨. 이후 효율적인 연산을 위해 torch.FloatTensor 배열로 바꾸어야 하는데, 이때 픽셀 값의 범위는 [0.0, 1.0] 사이가 되고 차원의 순서도 (채널 수 C×높이 H×너비 W)로 바뀜. 그리고 이러한 작업을 수행해 주는 메서드가 ToTensor()임.

(e) `transforms.Normalize` : 전이 학습에서 사용하는 사전 훈련된 모델들은 대개 ImageNet 데이터셋에서 훈련되었음. 따라서 사전 훈련된 모델을 사용하기 위해서는 ImageNet 데이터의 각 채널별 평균과 표준편차에 맞는 정규화(normalize)¹를 해 주어야 함. 즉, Normalize 메서드 안에 사용된 (mean: 0.485, 0.456, 0.406), (std: 0.229, 0.224, 0.225)는 ImageNet에서 이미지들의 RGB 채널마다 평균과 표준편차를 의미함. 참고로 OpenCV를 사용해서 이미지를 읽어 온다면 RGB 이미지가 아닌 BGR 이미지이므로 채널 순서에 주의해야 함.

¹ 데이터 범위를 사용자가 원하는 범위로 제한하는 것을 의미함. 예를 들어 이미지 데이터를 0~1.0 사이의 값을 갖도록 하는 것이 정규화임. 자세한 내용은 8장을 참고함.

② `__call__` 함수는 클래스를 호출할 수 있도록 하는 메서드임. `__init__` 은 인스턴스 초기화를 위해 사용한다면 `__call__` 은 인스턴스가 호출되었을 때 실행됨. 즉, 클래스에 `__call__` 함수가 있을 경우 클래스 객체 자체를 호출하면 `__call__` 함수의 리턴(return) 값이 반환됨.

이미지가 위치한 디렉터리에서 데이터를 불러온 후 훈련용으로 400개의 이미지, 검증용으로 92개의 이미지, 테스트용으로 열 개의 이미지를 사용함.

예제를 진행하기 위한 시스템 성능이 좋을 경우 훈련용 2만 개를 사용하면 모델 성능이 더 높아짐. 예제 파일의 주석(#)을 해제하고 실행하면 됨. 단 해제하고 실행한다면 그다음 두 줄은 주석 처리해야 함.

코드 6-3 이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리

```
cat_directory = r'../chap06/data/dogs-vs-cats/Cat/'
dog_directory = r'../chap06/data/dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in
                                os.listdir(cat_directory)]) ----- ①
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in
                                os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths] ----- ②
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None] ----- ②

random.seed(42) ----- ③
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] ----- 훈련용 400개의 이미지
val_images_filepaths = correct_images_filepaths[400:-10] ----- 검증용 92개의 이미지
test_images_filepaths = correct_images_filepaths[-10:] ----- 테스트용 10개의 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

① 고양이 이미지 데이터를 가져옴.

```
cat_images_filepaths = sorted([os.path.join(cat_directory, f)
                                for f in os.listdir(cat_directory)])
                                (a) (b) (c)
```

① sorted: 데이터를 정렬된 리스트로 만들어서 반환함.

② os.path.join: 경로와 파일명을 결합하거나 분할된 경로를 하나로 합치고 싶을 때 사용함. 즉, cat_directory 디렉터리(../chap06/data/dogs-vs-cats/Cat/)와 os.listdir을 통해 검색된 이미지 파일들(f)을 하나로 합쳐서 '../chap06/data/dogs-vs-cats/Cat/이미지 파일명'(예 ../chap06/data/dogs-vs-cats/Cat/cat.0.jpg)으로 표시해 줌.

또한, 다음과 같이 경로를 하나로 합칠 수 있음.

```
import os
list_path = ['C:\\', 'Temp', 'user']
folder_path = os.path.join(*list_path)
folder_path
```

다음은 실행 결과임. 다음과 같이 경로가 하나로 합쳐 있는 것을 확인할 수 있음.

'C:\\Temp\\user'

참고로 윈도 환경에서는 경로가 '\\'으로 표시됨.

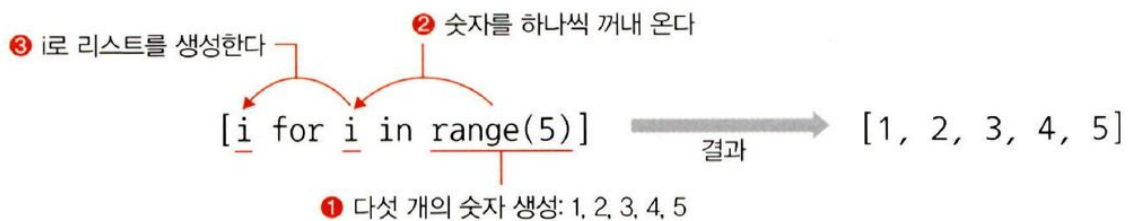
③ `os.listdir`: 지정한 디렉터리 내 모든 파일의 리스트를 반환함. 예제에서 사용하는 Cat 디렉터리의 이미지 파일들을 모두 반환함.

② `images_filepaths`에서 이미지 파일들을 불러옴.

```
correct_images_filepaths = [i for i in images_filepaths  
                             (a) (b)  
                             if cv2.imread(i) is not None]  
                             (c)
```

① for 반복문을 이용하여 가져온 데이터에 대해 i를 이용하여 리스트로 만들. 즉, 그림 6-3과 같은 의미를 갖음.

♥ 그림 6-3 for 반복문



② 반복문(for)을 이용하여 `images_filepaths`에서 이미지 데이터를 검색함.

③ 조건문(if)을 의미함. `cv2.imread()` 함수(`cv2.imread(i)`)를 이용하여 모든 이미지 데이터를 읽어 옴(not None 상태, 즉 '더 이상 데이터를 찾을 수 없을 때까지'를 의미함).

③ 넘파이 `random()` 함수는 임의의 난수를 생성하는데, 이때 난수를 생성하기 위해 사용되는 것이 시드 값(seed value)임. 또한, `Numpy.random.seed()` 메서드는 상태를 초기화함. 즉, 이 모듈이 호출될 때마다 임의의 난수가 재생성됨. 하지만 특정 시드 값을 부여하면 상태가 저장되기 때문에 동일한 난수를 생성함. 예를 들어 다음과 같음.

```
import numpy as np
```

```
np.random.seed(101)
```

```
np.random.randint(low=1, high=10, size=10)
```

시드 값을 101로 설정했을 때의 결과는 다음과 같음.

```
array([2, 7, 8, 9, 5, 9, 6, 1, 6, 9])
```

이번에는 시드 값을 100으로 설정해 보겠음.

```
np.random.seed(100)  
np.random.randint(low=1, high=10, size=10)
```

시드 값을 100으로 설정했을 때의 결과는 다음과 같음.

```
array([9, 9, 4, 8, 8, 1, 5, 3, 6, 3])
```

시드 값을 101로 설정했을 때와는 다른 결과를 보여 줌.

또다시 시드 값을 101로 설정하겠음.

```
np.random.seed(101)  
np.random.randint(low=1, high=10, size=10)
```

시드 값을 101로 설정했더니 앞에서와 같이 동일한 값이 출력된 것을 알 수 있음.

```
array([2, 7, 8, 9, 5, 9, 6, 1, 6, 9])
```

주어진 데이터셋을 훈련, 검증, 테스트 용도로 분리했는데, 테스트 용도의 데이터셋에 어떤 데이터들이 있는지 확인해 보겠음.

코드 6-4 테스트 데이터셋 이미지 확인 함수

```
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) ----- ①
        true_label = os.path.normpath(image_filepath).split(os.sep)[-2] ----- ②
        predicted_label = predicted_labels[i] if predicted_labels else true_label ----- ③
        color = "green" if true_label == predicted_label else "red" -----
        ax.ravel()[i].imshow(image) ----- 개별 이미지를 출력
        ax.ravel()[i].set_title(predicted_label, color=color) ----- predicted_label을
        ax.ravel()[i].set_axis_off() ----- 이미지의 축 제거 타이틀로 사용
    plt.tight_layout() ----- 이미지의 여백을 조정 예측과 정답(레이블)이 동일하면 초록색으로
    plt.show() 표시하고, 그렇지 않다면 빨간색으로 표시
```

① cv2.cvtColor는 이미지의 색상을 변경하기 위해 사용되며 파라미터는 다음과 같음.

`cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`

Ⓐ

Ⓑ

Ⓐ 첫 번째 파라미터: 입력 이미지

Ⓑ 두 번째 파라미터: 변환할 이미지의 색상을 지정하는 것으로 BGR(Blue, Green, Red) 채널 이미지를 RGB(컬러)로 변경하겠다는 의미임.

② 이미지의 전체 경로를 정규화하고 분할을 위한 코드로 파라미터는 다음과 같음.

`true_label = os.path.normpath(image_filepath).split(os.sep)[-2]`

Ⓐ

Ⓑ

Ⓐ `os.path.normpath`: 경로명을 정규화함. 예를 들어 A//B, A/B/, A/.B 및 A/./B 모두 A/B로 경로를 통일함.

Ⓑ `split(os.sep)`: 경로를 / 혹은 \를 기준으로 분할할 때 사용함. 예를 들어 c:/temp/user/a.jpg라는 경로가 있을 때 `split(os.sep)`를 적용하면 ['c:', 'temp', 'user', 'a.jpg']처럼 분할됨. 또한, `split(os.sep)[-2]`를 적용하면 'user'를 반환할 것임.

③ `predicted_label`에 대한 값을 정의하기 위한 코드임.

`predicted_label = predicted_labels[i] if predicted_labels else true_label`

Ⓐ

Ⓑ

Ⓐ `predicted_labels` 값이 있으면 그 값을 `predicted_label`로 사용함.

Ⓑ `predicted_labels`에 값이 없다면 `true_label` 값을 `predicted_label`로 사용함.

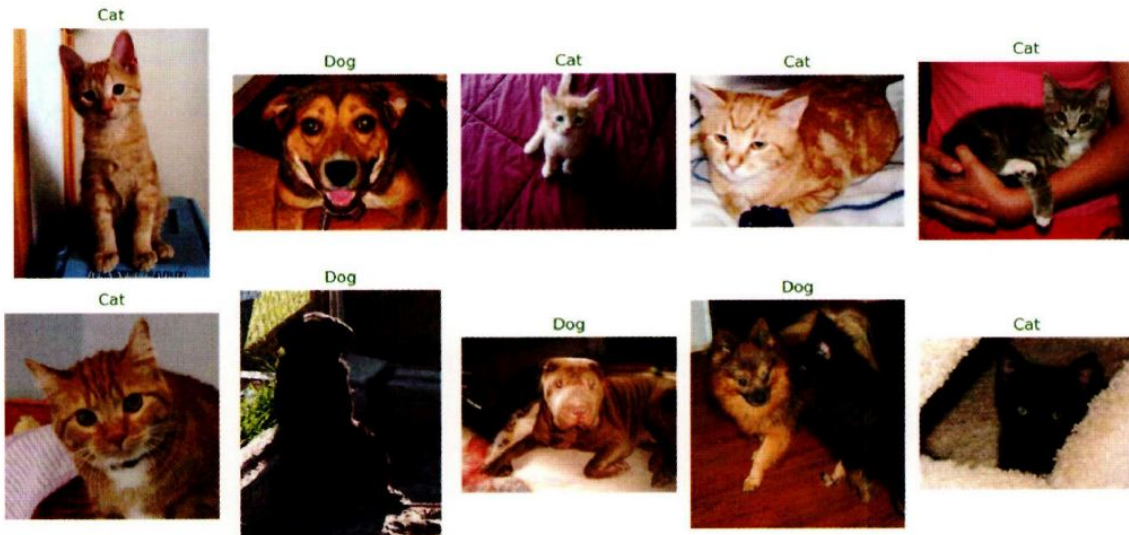
앞에서 정의한 함수를 호출하여 열 개의 이미지를 출력함.

코드 6-5 테스트 데이터셋 이미지를 출력

```
display_image_grid(test_images_filepaths)
```

다음 그림은 테스트 용도의 데이터셋 열 개를 출력한 결과임.

♥ 그림 6-4 테스트 데이터셋의 이미지



기본적인 데이터셋 준비가 완료되었음. 이제부터 모델 학습을 위한 구체적인 단계들이 시작될 텐데 순서는 다음과 같음. 먼저 데이터셋²에는 학습할 데이터의 경로를 정의하고 그 경로에서 데이터를 읽어 올. 데이터셋 크기가 클 수 있으므로 `init` 에서 전체 데이터를 읽어 오는 것이 아니라 경로만 저장해 놓고, `getitem` 메서드에서 이미지를 읽어 올. 즉, 데이터를 어디에서 가져올지 결정함. 이후 데이터로더에서 데이터셋의 데이터를 메모리로 불러오는데, 한꺼번에 전체 데이터를 불러오는 것이 아니라 배치 크기만큼 분할하여 가져옴.

² 여기에서 데이터셋은 `torch.utils.data.Dataset`을 의미함.

이번에 살펴볼 `DogvsCatDataset()` 클래스는 데이터를 불러오는 방법을 정의함. 이번 예제의 목적은 다수의 개와 고양이 이미지가 포함된 데이터에서 이들을 예측하는 것임. 따라서 레이블(정답) 이미지에서 고양이와 개가 포함될 확률을 코드로 구현함. 예를 들어 고양이가 있는 이미지의 레이블은 0이 되고, 개가 있는 이미지의 레이블은 1이 되도록 코드를 구현함.

코드 6-6 이미지 데이터셋 클래스 정의

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path)
        img_transformed = self.transform(img, self.phase)
        label = img_path.split('/')[-1].split('.')[0]
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0
        return img_transformed, label
```

데이터셋의 전처리 (데이터 변형 적용)

DogvsCatDataset 클래스를 호출할 때 transform에 대한 매개변수를 받아 옵니다.

images_filepaths 데이터셋의 전체 길이를 반환

데이터셋에서 데이터를 가져오는 부분으로 결과는 텐서 형태가 됩니다.

img_path 위치에서 이미지 데이터들을 가져옵니다.

이미지에 'train' 전처리를 적용

①

① 이미지 데이터에 대한 레이블 값(dog, cat)을 가져옴.

```
label = img_path.split('/')[-1].split('.')[0]
```

(a) (b) (c)

① img_path: 이미지가 위치한 전체 경로를 보여 줌(필자가 사용한 경로는 다음과 같지만 환경에 따라 경로는 다를 수 있음).

예 `E:/torch/chap06/data/dogs-vs-cats/Dog/dog.113.jpg`

이때 이미지 데이터의 레이블(dog)을 가져오려면 '/'와 '.'을 제거해야 함. 다음의 ②와 ③가 '/' , '.'을 제거하기 위해 사용됨.

② `img_path.split('/')[-1]`: 이미지의 전체 경로에서 '/'를 제거함. 제거 결과는 다음과 같음.

예 `dog.113.jpg`

③ `split('.')[0]`: 마지막으로 `img_path.split('/')[-1]` 을 통해 얻은 결과인 'dog.113.jpg'에서 '.'을 제거함. 이때 'dog.113.jpg'를 '.'을 기준으로 분리시켰고, 분리된 값들에서 첫 번째 값([0])을 가져오면 결과는 다음과 같음.

예 `dog`

전처리에서 사용할 변수에 대한 값을 정의함.

코드 6-7 변수 값 정의

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

훈련과 검증 용도의 데이터셋을 정의함. 앞에서 정의한 DogvsCatDataset() 클래스를 이용하여 훈련과 검증 데이터셋을 준비하되 전처리도 함께 적용하도록 함.

코드 6-8 이미지 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size,
                                                                              mean, std), phase='train') ----- 훈련 이미지에 train_transforms를 적용
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size,
                                                                              mean, std), phase='val') ----- 검증 이미지에 test_transforms를 적용

index = 0
print(train_dataset.__getitem__(index)[0].size()) ----- 훈련 데이터(train_dataset.__getitem__[0][0])의 크기(size()) 출력
print(train_dataset.__getitem__(index)[1]) ----- 훈련 데이터의 레이블 출력
```

다음은 훈련 데이터의 크기와 레이블에 대한 출력 결과임. 데이터가 랜덤으로 섞여 있는 상태이기 때문에 책과 다른 결과가 나올 수 있음.

```
torch.Size([3, 224, 224])
1
```

이미지는 컬러 상태에서 224×224 크기를 가지며 레이블이 1로 출력되었음. 즉, 훈련 데이터셋의 레이블 (`train_dataset.__getitem__(index)[1]`)이 1 값을 갖기 때문에 '개'라는 이미지가 포함되어 있다는 것을 유추해 볼 수 있음.

전처리와 함께 데이터셋을 정의했기 때문에 이제 메모리로 불러와서 훈련을 위한 준비를 함.

코드 6-9 데이터로더 정의

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True) ----- ①
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader} -----
batch_iterator = iter(train_dataloader) ----- 훈련 데이터셋(train_dataloader)과 검증
inputs, label = next(batch_iterator) ----- 데이터셋(val_dataloader)을 합쳐서 표현
print(inputs.size())
print(label)
```

① 파이토치의 데이터로더는 배치 관리를 담당함. 한 번에 모든 데이터를 불러오면 메모리에 부담을 줄 수 있기 때문에 데이터를 그룹으로 쪼개서 조금씩 불러옴.

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

(a) (b) (c)

① 첫 번째 파라미터: 데이터를 불러오기 위한 데이터셋임.

② `batch_size`: 한 번에 메모리로 불러올 데이터 크기로, 여기에서는 32개씩 데이터를 가져옴.

③ `shuffle`: 메모리로 데이터를 가져올 때 임의로 섞어서 가져오도록 함.

다음은 데이터로더를 이용하여 훈련 데이터셋을 메모리로 불러온 후 데이터셋의 크기와 레이블을 출력한 결과임.

```
torch.Size([32, 3, 224, 224])
tensor([1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1,
0, 0, 0, 0, 1, 0])
```

이제 데이터셋을 학습시킬 모델의 네트워크를 설계하기 위한 클래스를 생성함.

코드 6-10 모델의 네트워크 클래스

```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
```

```

                2D 합성곱층이 적용됩니다. 이때 입력 형태는 (3, 224, 224)가 되며 출력
                형태는 (weight-kernel_size+1)/stride에 따라 (16, 220, 220)이 됩니다.
self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=5, stride=1,-----
                    padding=0)
self.relu1 = nn.ReLU() ----- ReLU 활성화 함수입니다. ----- 최대 풀링이 적용됩니다. 적용 이후 출력 형태는
self.maxpool1 = nn.MaxPool2d(kernel_size=2) ----- 220/2가 되어 (16, 110, 110)입니다.
self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5,
                    stride=1, padding=0) ----- 또다시 2D 합성곱층이 적용되며 출력 형태는
self.relu2 = nn.ReLU() ----- (32, 106, 106)입니다.
self.maxpool2 = nn.MaxPool2d(kernel_size=2) ----- 최대 풀링이 적용되며 출력 형태는
                    (32, 53, 53)입니다.

self.fc1 = nn.Linear(32*53*53, 512)
self.relu5 = nn.ReLU()
self.fc2 = nn.Linear(512, 2)
self.output = nn.Softmax(dim=1)

def forward(self, x):
    out = self.cnn1(x)
    out = self.relu1(out)
    out = self.maxpool1(out)
    out = self.cnn2(out)
    out = self.relu2(out)
    out = self.maxpool2(out)
    out = out.view(out.size(0), -1) ----- 완전연결층에 데이터를 전달하기 위해
    out = self.fc1(out) ----- 데이터 형태를 1차원으로 바꿉니다.
    out = self.fc2(out)
    out = self.output(out)
    return out

```

네트워크의 각 부분들을 통과할 때마다 입력과 출력의 형태가 바뀌는데, 그 계산은 다음 수식과 같음(수식은 5장에 자세히 설명되어 있음).

Conv2d 계층에서의 출력 크기 구하는 공식

출력 크기 = $(W - F + 2P) / S + 1$

- W: 입력 데이터의 크기(input_volume_size)
- F: 커널 크기(kernel_size)
- P: 패딩 크기(padding_size)
- S: 스트라이드(strides)

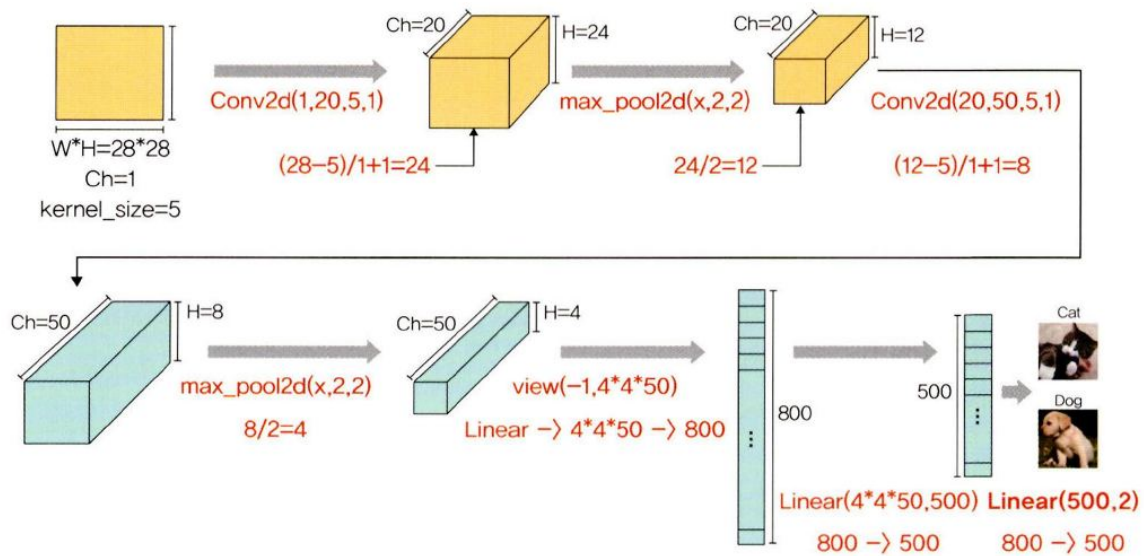
MaxPool2d 계층에서의 출력 크기 구하는 공식

출력 크기 = IF / F

- IF: 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 함)
- F: 커널 크기(kernel_size)

각각의 계층에 대한 결과는 다음 그림과 같음.

▼ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



LeNet()을 model이라는 이름으로 객체를 생성하여 모델 학습을 위한 준비를 함.

코드 6-11 모델 객체 생성

```
model = LeNet()
print(model)
```

다음은 생성한 model에 대한 출력 결과임.

```
LeNet(
  (cnn1): Conv2d(3, 16, kernel_size=(5, 5), stride=(1, 1))
  (relu1): ReLU()
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn2): Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1))
  (relu2): ReLU()
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=89888, out_features=512, bias=True)

  (relu5): ReLU()
  (fc2): Linear(in_features=512, out_features=2, bias=True)
  (output): Softmax(dim=1)
)
```

출력 결과가 한눈에 들어오지 않는다면 torchsummary 라이브러리를 사용해 볼 수 있음. torchsummary는 케라스와 같은 형태로 모델을 출력해 볼 수 있는 라이브러리임.

ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0

=====
Total params: 46,038,242

Trainable params: 46,038,242

Non-trainable params: 0

Input size (MB): 0.57

Forward/backward pass size (MB): 19.47

Params size (MB): 175.62

Estimated Total Size (MB): 195.67

다음은 모델의 학습 가능한 파라미터 수를 `model.parameters()`를 이용하여 확인해 보겠음. 전반적인 코드의 흐름과는 관련이 없는 부분이지만 정보 확인을 위한 코드임. 참고로 해당 정보는 앞에서 실행했던 `summary`로도 확인이 가능함.

코드 6-13 학습 가능한 파라미터 수 확인

```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'The model has {count_parameters(model):,} trainable parameters')
```

다음 실행 결과를 보니 46,038,242개의 파라미터를 학습해야 함을 확인할 수 있음. 꽤 많은 파라미터를 학습해야 하기 때문에 추후 데이터를 모델에 적용했을 때 긴 시간이 걸릴 것을 예상할 수 있음. 그런 이유로 캐글에서 내려받은 이미지 전체를 사용하는 것이 아닌 일부 이미지만 예제에서 사용함.

The model has 46,038,242 trainable parameters

모델의 네트워크 구성이 완료되었기 때문에 이제 옵티마이저와 손실 함수를 정의함.

코드 6-14 옵티마이저와 손실 함수 정의

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1

_____ () _____ () _____ ()

[illegible]

○ 중요한 개념 : 가장자리 사회 (가장자리 사회)를 정의하는 개념이다.

[illegible]

※ 1. 2. 3. 4. 5. 6. 7. 8. 9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100. 101. 102. 103. 104. 105. 106. 107. 108. 109. 110. 111. 112. 113. 114. 115. 116. 117. 118. 119. 120. 121. 122. 123. 124. 125. 126. 127. 128. 129. 130. 131. 132. 133. 134. 135. 136. 137. 138. 139. 140. 141. 142. 143. 144. 145. 146. 147. 148. 149. 150. 151. 152. 153. 154. 155. 156. 157. 158. 159. 160. 161. 162. 163. 164. 165. 166. 167. 168. 169. 170. 171. 172. 173. 174. 175. 176. 177. 178. 179. 180. 181. 182. 183. 184. 185. 186. 187. 188. 189. 190. 191. 192. 193. 194. 195. 196. 197. 198. 199. 200. 201. 202. 203. 204. 205. 206. 207. 208. 209. 210. 211. 212. 213. 214. 215. 216. 217. 218. 219. 220. 221. 222. 223. 224. 225. 226. 227. 228. 229. 230. 231. 232. 233. 234. 235. 236. 237. 238. 239. 240. 241. 242. 243. 244. 245. 246. 247. 248. 249. 250. 251. 252. 253. 254. 255. 256. 257. 258. 259. 260. 261. 262. 263. 264. 265. 266. 267. 268. 269. 270. 271. 272. 273. 274. 275. 276. 277. 278. 279. 280. 281. 282. 283. 284. 285. 286. 287. 288. 289. 290. 291. 292. 293. 294. 295. 296. 297. 298. 299. 300. 301. 302. 303. 304. 305. 306. 307. 308. 309. 310. 311. 312. 313. 314. 315. 316. 317. 318. 319. 320. 321. 322. 323. 324. 325. 326. 327. 328. 329. 330. 331. 332. 333. 334. 335. 336. 337. 338. 339. 340. 341. 342. 343. 344. 345. 346. 347. 348. 349. 350. 351. 352. 353. 354. 355. 356. 357. 358. 359. 360. 361. 362. 363. 364. 365. 366. 367. 368. 369. 370. 371. 372. 373. 374. 375. 376. 377. 378. 379. 380. 381. 382. 383. 384. 385. 386. 387. 388. 389. 390. 391. 392. 393. 394. 395. 396. 397. 398. 399. 400. 401. 402. 403. 404. 405. 406. 407. 408. 409. 410. 411. 412. 413. 414. 415. 416. 417. 418. 419. 420. 421. 422. 423. 424. 425. 426. 427. 428. 429. 430. 431. 432. 433. 434. 435. 436. 437. 438. 439. 440. 441. 442. 443. 444. 445. 446. 447. 448. 449. 450. 451. 452. 453. 454. 455. 456. 457. 458. 459. 460. 461. 462. 463. 464. 465. 466. 467. 468. 469. 470. 471. 472. 473. 474. 475. 476. 477. 478. 479. 480. 481. 482. 483. 484. 485. 486. 487. 488. 489. 490. 491. 492. 493. 494. 495. 496. 497. 498. 499. 500. 501. 502. 503. 504. 505. 506. 507. 508. 509. 510. 511. 512. 513. 514. 515. 516. 517. 518. 519. 520. 521. 522. 523. 524. 525. 526. 527. 528. 529. 530. 531. 532. 533. 534. 535. 536. 537. 538. 539. 540. 541. 542. 543. 544. 545. 546. 547. 548. 549. 550. 551. 552. 553. 554. 555. 556. 557. 558. 559. 560. 561. 562. 563. 564. 565. 566. 567. 568. 569. 570. 571. 572. 573. 574. 575. 576. 577. 578. 579. 580. 581. 582. 583. 584. 585. 586. 587. 588. 589. 590. 591. 592. 593. 594. 595. 596. 597. 598. 599. 600. 601. 602. 603. 604. 605. 606. 607. 608. 609. 610. 611. 612. 613. 614. 615. 616. 617. 618. 619. 620. 621. 622. 623. 624. 625. 626. 627. 628. 629. 630. 631. 632. 633. 634. 635. 636. 637. 638. 639. 640. 641. 642. 643. 644. 645. 646. 647. 648. 649. 650. 651. 652. 653. 654. 655. 656. 657. 658. 659. 660. 661. 662. 663. 664. 665. 666. 667. 668. 669. 670. 671. 672. 673. 674. 675. 676. 677. 678. 679. 680. 681. 682. 683. 684. 685. 686. 687. 688. 689. 690. 691. 692. 693. 694. 695. 696. 697. 698. 699. 700. 701. 702. 703. 704. 705. 706. 707. 708. 709. 710. 711. 712. 713. 714. 715. 716. 717. 718. 719. 720. 721. 722. 723. 724. 725. 726. 727. 728. 729. 730. 731. 732. 733. 734. 735. 736. 737. 738. 739. 740. 741. 742. 743. 744. 745. 746. 747. 748. 749. 750. 751. 752. 753. 754. 755. 756. 757. 758. 759. 760. 761. 762. 763. 764. 765. 766. 767. 768. 769. 770. 771. 772. 773. 774. 775. 776. 777. 778. 779. 780. 781. 782. 783. 784. 785. 786. 787. 788. 789. 790. 791. 792. 793. 794. 795. 796. 797. 798. 799. 800. 801. 802. 803. 804. 805. 806. 807. 808. 809. 810. 811. 812. 813. 814. 815. 816. 817. 818. 819. 820. 821. 822. 823. 824. 825. 826. 827. 828. 829. 830. 831. 832. 833. 834. 835. 836. 837. 838. 839. 840.

코드 6-15 모델의 파라미터와 손실 함수를 CPU에 할당

[illegible]

코드 6-16 모델 학습 함수 정의

```

if phase == 'train':
    model.train() ----- 모델을 학습시키겠다는 의미
else:
    model.eval()

epoch_loss = 0.0
epoch_corrects = 0

for inputs, labels in tqdm(dataloader_dict[phase]): ----- 여기서 dataloader_dict는
    inputs = inputs.to(device) ----- 훈련 데이터셋을 CPU에 할당
    labels = labels.to(device)
    optimizer.zero_grad() ----- 역전파 단계를 실행하기 전에 기울기(gradient)를 0으로 초기화
    의미

    with torch.set_grad_enabled(phase == 'train'):
        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)
        loss = criterion(outputs, labels) ----- 손실 함수를 이용한 오차 계산

        if phase == 'train':
            loss.backward() ----- 모델의 학습 가능한 모든 파라미터에 대해 기울기를 계산
            optimizer.step() ----- optimizer의 step 함수를 호출하면 파라미터를 갱신
            ----- 정답과 예측이 일치하면 그것의 합계를 epoch_corrects에 저장
            epoch_loss += loss.item() * inputs.size(0) ----- ①
            epoch_corrects += torch.sum(preds == labels.data) -----
            ----- 최종 오차 계산(오차를 데이터셋의 길이(개수)로 나누어서 계산)
epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset) -----
epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset) -----
            ----- 최종 정확도(epoch_corrects를 데이터셋의 길이(개수)로 나누어서 계산)
print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

if phase == 'val' and epoch_acc > best_acc: ----- 검증 데이터셋에 대한 가장 최적의
    best_acc = epoch_acc ----- 정확도를 저장
    best_model_wts = model.state_dict()

time_elapsed = time.time() - since
print('Training complete in {:.0f}m {:.0f}s'.format(
    time_elapsed // 60, time_elapsed % 60))
print('Best val Acc: {:.4f}'.format(best_acc))
return model

```

① 오차와 입력을 곱하는 이유를 알아보기 위해 손실 함수의 reduction이라는 파라미터를 이해할 필요가 있음.

예를 들어 코드 6-14에서 손실 함수를 정의할 때 다음과 같이 지정할 수 있음. reduction을 따로 명시하지 않았던 이유는 기본값을 그대로 사용하기 때문임.

```
criterion = nn.CrossEntropyLoss(reduction='mean')
```

reduction 파라미터의 기본값은 'mean'임. 'mean'은 정답과 예측 값의 오차를 구한 후 그 값들의 평균을 반환함. 즉, 손실 함수 특성상 전체 오차를 배치 크기로 나눴으로써 평균을 반환하기 때문에 epoch_loss를 계산하는 동안 loss.item()과 inputs.size(0)을 곱해 줌.

이제 모델을 학습시키기 위해 train_model을 호출함.

코드 6-17 모델 학습

```
import time

num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)
```

다음은 모델 학습 결과임.

Epoch 1/10

100%

13/13 [00:20<00:00, 1.40s/it]

train Loss: 0.6971 Acc: 0.4875

100%

3/3 [00:02<00:00, 1.15it/s]

val Loss: 0.7125 Acc: 0.4457

Epoch 2/10

100%

13/13 [00:21<00:00, 1.64s/it]

train Loss: 0.6890 Acc: 0.5200

100%

3/3 [00:04<00:00, 1.50s/it]

val Loss: 0.6837 Acc: 0.5978

Epoch 3/10

... 중간 생략 ...

100%

13/13 [00:20<00:00, 1.43s/it]

```
train Loss: 0.6531 Acc: 0.6450
100%
3/3 [00:02<00:00, 1.25it/s]
val Loss: 0.6569 Acc: 0.5870
Epoch 10/10
-----
100%
13/13 [00:21<00:00, 1.48s/it]
train Loss: 0.6562 Acc: 0.5975
100%
3/3 [00:02<00:00, 1.23it/s]
val Loss: 0.6507 Acc: 0.6304
Training complete in 4m 6s
Best val Acc: 0.641304
```

검증 데이터셋을 이용한 모델 학습 결과 최고 64%의 정확도를 보이고 있음. 높지 않은 결과라고 할 수 있음. 좀 더 정확한 결과를 원한다면 데이터셋을 늘려서 테스트해 보길 권장함.

훈련 데이터셋과 더불어 테스트 데이터셋을 모델에 적용하여 정확도를 측정해 보겠음. 측정 결과는 데이터 프레임에 담아 둔 후 CSV 파일로 저장함. 테스트 용도의 데이터셋을 이용하므로 `model.eval()`을 사용함.

코드 6-18 모델 테스트를 위한 함수 정의

```
import pandas as pd

id_list = []
pred_list = []
_id = 0
with torch.no_grad(): ..... 역전파 중 텐서들에 대한 변화도를 계산할 필요가 없음을 나타내는 것으로,
                               ..... 훈련 데이터셋의 모델 학습과 가장 큰 차이점입니다.
    for test_path in tqdm(test_images_filepaths): ..... 테스트 데이터셋 이용
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val') ..... 테스트 데이터셋 전처리 적용
        img = img.unsqueeze(0) ..... ①
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs, dim=1)[: , 1].tolist() ..... ②
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
}) ..... 테스트 데이터셋의 예측 결과인 id와 레이블(label)을 데이터 프레임에 저장

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('../chap06/data/LeNet', index=False) ..... 데이터 프레임을 CSV 파일로 저장
```

① torch.unsqueeze는 텐서에 차원을 추가할 때 사용함. 또한, (0)은 차원이 추가될 위치를 의미함.

예를 들어 형태가 (3)인 텐서가 있다고 가정해 보겠음. 0 위치에 차원을 추가하면 형태가 (1,3)이 됨. 즉, 행 한 개와 열 세 개의 구조를 갖는 텐서가 만들어짐. 그렇다면 2D에 텐서가 추가될 때는 어떨까?

- 형태가 (2,2)인 2D 텐서가 있을 때 0 위치에 차원을 추가하면 텐서 모양이 (1,2,2)가 됨. 이는 하나의 채널, 행 두 개와 열 두 개를 의미함.
- 1 위치에 차원을 추가하면 (2,1,2)의 형태가 되므로 채널 두 개, 행 한 개, 열 두 개가 됨.
- 2 위치에 차원을 추가하면 텐서는 (2,2,1)의 형태가 됨. 즉, 채널 두 개와 행 두 개, 열 한 개를 의미함.

② 소프트맥스(softmax)는 지정된 차원(dim)을 따라 텐서의 요소(텐서의 개별 값)가 (0, 1) 범위에 있고 합계가 1이 되도록 크기를 다시 조정함.

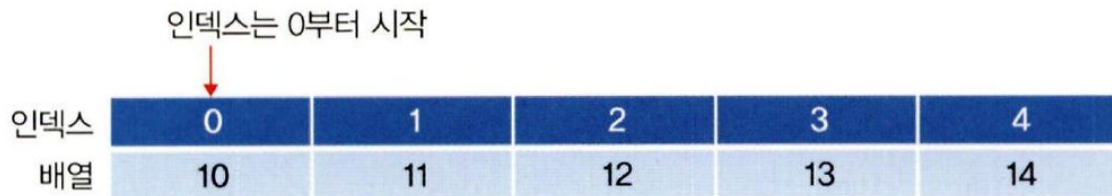
`F.softmax(outputs, dim=1)[[:, 1]].tolist()`

(a) (b) (c)

- ① outputs에 softmax를 적용하여 각 행의 합이 1이 되도록 함.
- ② ①의 값 중 모든 행(:)에서 두 번째 칼럼(1번째 인덱스)을 가져옴.

♥ 그림 6-6 배열과 인덱스

인덱스는 0부터 시작



인덱스	0	1	2	3	4
배열	10	11	12	13	14

- ③ 배열을 리스트 형태로 변환함.

다음은 모델 예측 함수를 실행한 결과임. 예측 결과를 CSV 파일로 저장하는 함수였기 때문에 실행 결과는 큰 의미가 없으며 단순히 처리가 완료되었다고 이해하면 됨.

100%
10/10 [00:00<00:00, 22.46it/s]

테스트 데이터셋을 모델에 적용한 결과를 LeNet 파일로 저장해 두었음. 그 결과를 살펴봄.

코드 6-19 테스트 데이터셋의 예측 결과 호출

```
res.head(10)
```

다음은 DataFrame에서 열 개의 값을 보여 줌.

	id	label
0	115	0.412220
1	116	0.422235
2	162	0.468849
3	186	0.480325
4	190	0.350170
5	194	0.545277
6	207	0.550946
7	39	0.503651
8	42	0.507188
9	99	0.590979

예측 결과 레이블(label)이 0.5보다 크면 개를 의미하고, 0.5보다 작으면 고양이를 의미함.
 이제 마지막으로 예측 결과를 시각적으로 표현하기 위한 함수를 정의함.

코드 6-20 테스트 데이터셋 이미지를 출력하기 위한 함수 정의

```

class_ = classes = {0:'cat', 1:'dog'} ----- 개와 고양이에 대한 클래스 정의
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values) ----- 데이터 프레임의 id라는 컬럼에서 임의로 데이터를 가져옵니다.
        label = res.loc[res['id'] == a, 'label'].values[0]
        if label > 0.5: ----- 레이블 값이 0.5보다 크다면 개
            label = 1
        else: ----- 레이블 값이 0.5보다 작다면 고양이
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()

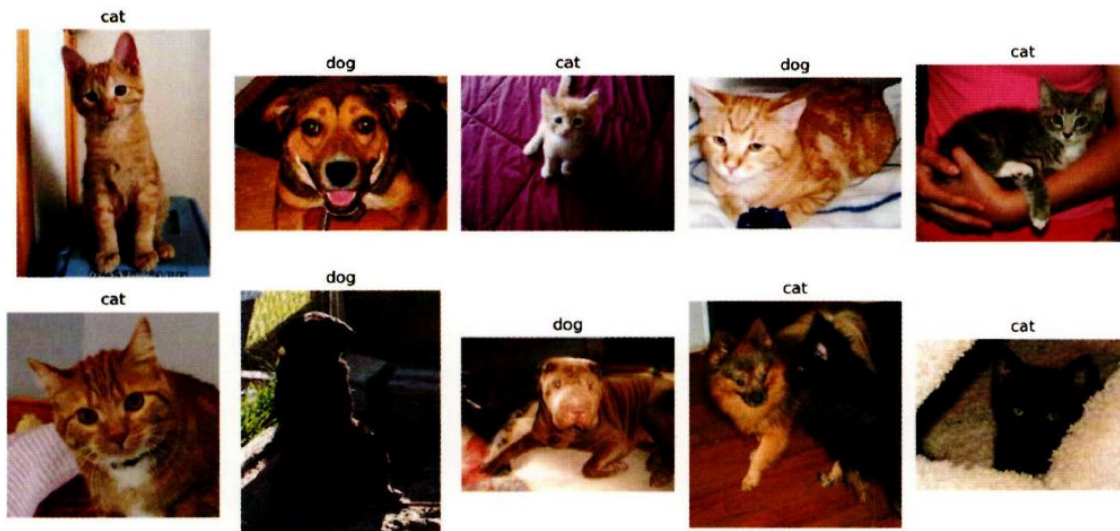
```

다음은 테스트 데이터셋에 대한 예측 결과를 시각적으로 보여 주기 위해 앞에서 정의한 함수를 호출함. 이때 전달되는 파라미터는 테스트 데이터셋임.

코드 6-21 테스트 데이터셋 예측 결과 이미지 출력

```
display_image_grid(test_images_filepaths)
```

♥ 그림 6-7 테스트 데이터셋에 대한 LeNet 모델의 예측 결과



예측력이 좋지는 않음. 극히 일부의 데이터를 이용한 모델 학습을 진행했기 때문에 이와 같은 예측력을 보여 줌. 앞에서 말했듯이 캐글에서 내려받은 데이터 전체를 사용한다면 성능이 높아지겠지만 CPU를 사용한다면 훈련 시간은 며칠이 걸릴 수도 있음.

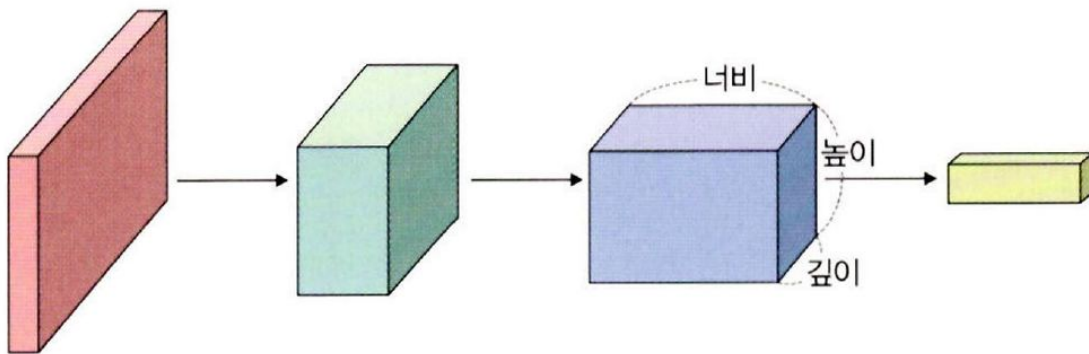
지금까지 LeNet의 코드를 살펴보고 이미지 분류에서 사용되는 또 다른 모델인 AlexNet에 대해 살펴보겠음. 이제부터 AlexNet을 포함한 여러 유형의 컴퓨터 비전 모델들을 살펴볼 텐데 LeNet에서 사용했던 코드와 유사한 부분이 많을 것임. 따라서 모델의 네트워크 위주로 CNN의 다양한 모델들을 살펴보면 학습에 도움이 많이 될 것임.

6.1.2 AlexNet

AlexNet은 ImageNet 영상 데이터베이스를 기반으로 한 화상 인식 대회인 'ILSVRC 2012'에서 우승한 CNN 구조임.

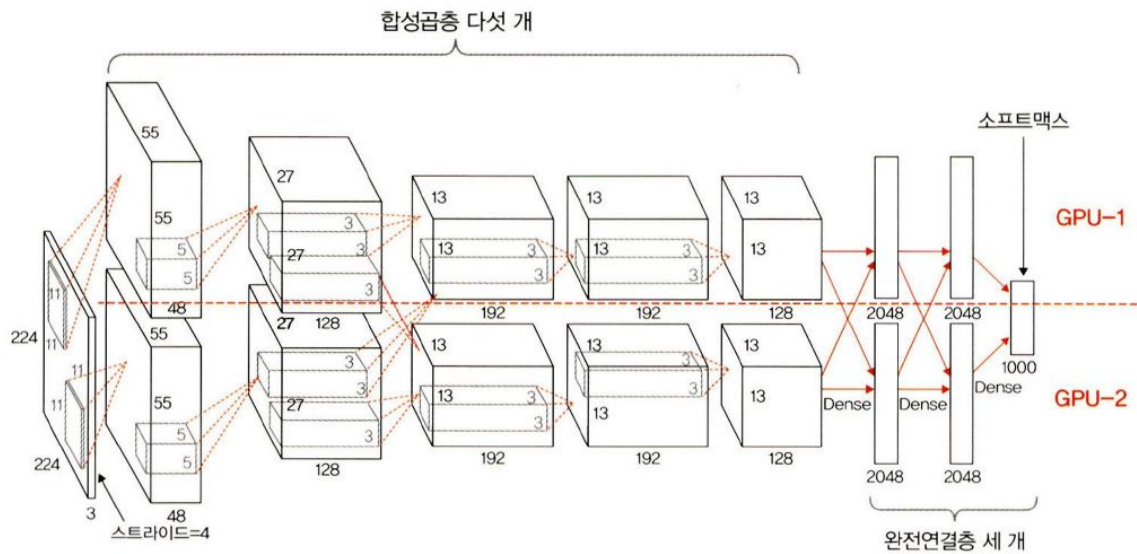
AlexNet을 설명하기에 앞서 AlexNet의 세부 블록을 이해하고자 CNN 구조를 다시 살펴봄. CNN은 다음 그림과 같이 3차원 구조를 갖는다는 것을 이해해야 함(이미지를 다루기 때문에 기본적으로 3차원 데이터를 다룸). 이미지 크기를 나타내는 너비(width)와 높이(height)뿐만 아니라 깊이(depth)를 갖음. 보통 색상이 많은 이미지는 R/G/B 성분 세 개를 갖기 때문에 시작이 3이지만, 합성곱을 거치면서 특성 맵이 만들어지고 이것에 따라 중간 영상의 깊이가 달라짐.

▼ 그림 6-8 CNN 구조



이것을 이해했다면 AlexNet 구조에 있는 숫자 의미에 대한 이해가 가능함. AlexNet은 합성곱층 총 다섯 개와 완전연결층 세 개로 구성되어 있으며, 맨 마지막 완전연결층은 카테고리 1000개를 분류하기 위해 소프트맥스 활성화 함수를 사용하고 있음. 전체적으로 보면 GPU 두 개를 기반으로 한 병렬 구조인 점을 제외하면 LeNet-5와 크게 다르지 않음.

♥ 그림 6-9 AlexNet 구조



AlexNet의 합성곱층에서 사용된 활성화 함수는 렐루(ReLU)로, 각 계층의 구조적 세부 사항은 다음 표를 참고함.

표 6-2 AlexNet 구조 상세

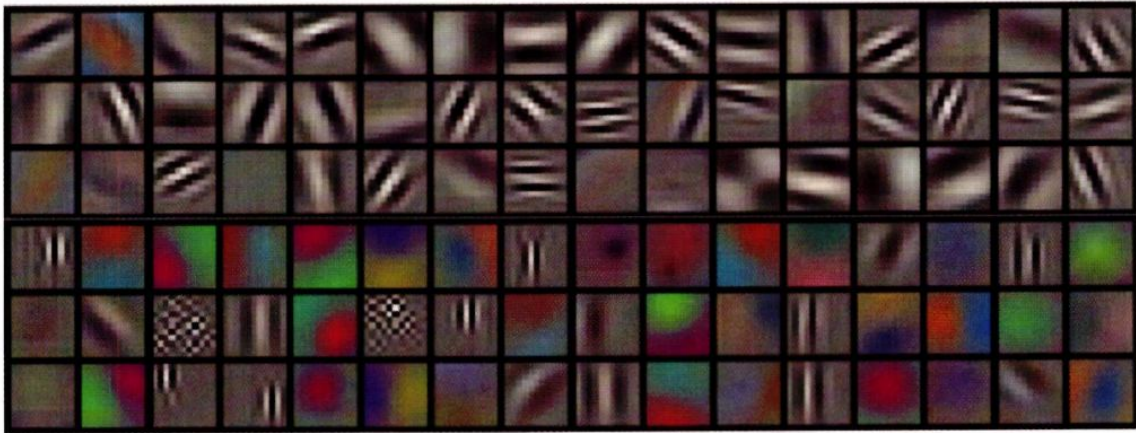
계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	–	–	–
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	–
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	–
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	–
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	1000	–	–	소프트맥스 (softmax)

네트워크에는 학습 가능한 변수가 총 6600만 개 있음. 네트워크에 대한 입력은 227×227×3 크기의 RGB 이미지이며, 각 클래스(혹은 카테고리)에 해당하는 1000×1 확률 벡터를 출력함.

AlexNet의 첫 번째 합성곱층 커널의 크기는 11×11×3이며, 스트라이드를 4로 적용하여 특성 맵을 96개 생성하기 때문에 55×55×96의 출력을 갖음. 첫 번째 계층을 거치면서 GPU-1에서는 주로 컬러와 상관없는 정보를 추출하기 위한 커널이 학습되고, GPU-2에서는 주로 컬러와 관련된 정보를 추출하기 위한 커널이 학습됨.

각 GPU의 결과가 다음 그림과 같이 나타남.

♥ 그림 6-10 AlexNet GPU-1 · 2 적용 결과



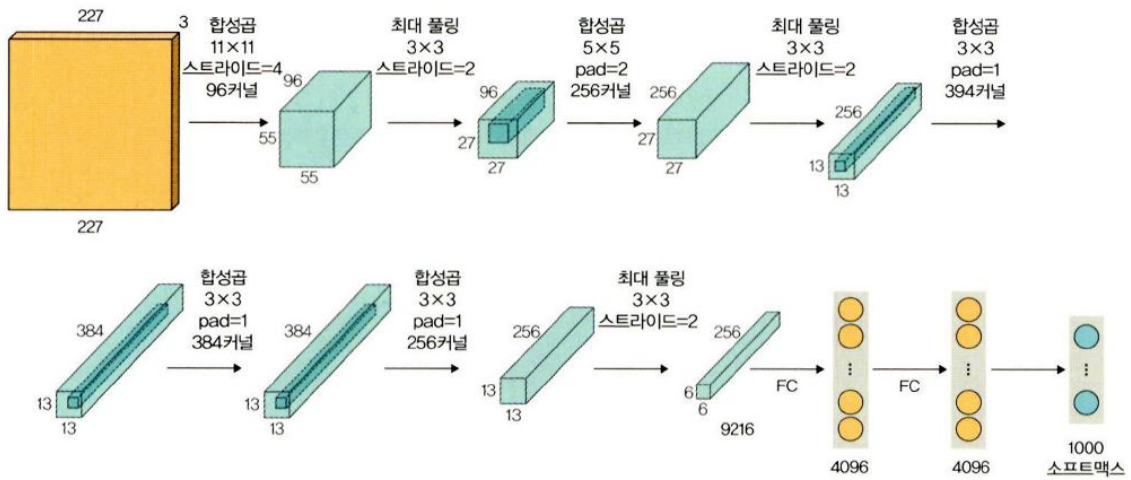
이제 파이토치 코드로 AlexNet에 대해 살펴봄. 먼저 필요한 라이브러리를 호출함. 전반적인 코드는 LeNet과 크게 다르지 않기 때문에 주로 모델을 구성하는 네트워크가 어떻게 차이가 나는지 위주로 살펴보면 좋음.

코드 6-22 필요한 라이브러리 호출

```
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from torch.autograd import Variable
from torch import optim
import torch.nn as nn
import torch.nn.functional as F
import os
import cv2
import random
from PIL import Image
from tqdm import tqdm_notebook as tqdm
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

데이터셋은 앞서 진행했던 개와 고양이 이미지를 계속 사용함. 이제 준비된 데이터를 이용할 네트워크를 생성해야 함. 원래 AlexNet의 마지막 계층에는 뉴런이 1000개 있지만 예제에서는 클래스 두 개만 사용함.

♥ 그림 6-11 AlexNet 예제 네트워크



데이터 전처리에 대한 코드임. 앞에서 이미 살펴보았기 때문에 설명은 생략함.

코드 6-23 데이터 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5,1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

이미지가 위치한 경로에서 데이터를 가져와 훈련, 검증, 테스트 용도로 분리함. 데이터가 위치한 경로에는 Cat과 Dog 폴더로 구성되어 있음.

코드 6-24 데이터를 가져와서 훈련, 검증, 테스트 용도로 분리

```
cat_directory = '../chap06/data/dogs-vs-cats/Cat/'
dog_directory = '../chap06/data/dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_
    directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.listdir(dog_
    directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400]
val_images_filepaths = correct_images_filepaths[400:-10]
test_images_filepaths = correct_images_filepaths[-10:]
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

다음은 훈련, 검증, 테스트에서 사용할 데이터셋의 이미지 수임.

400 92 10

우리가 사용하는 노트북(혹은 서버, 코랩)의 성능(CPU/GPU, 메모리)이 좋지 않다고 가정한 채 진행하고 있으므로 훈련 데이터셋을 400개로 제한했음.

AlexNet은 파라미터를 6000만 개 사용하는 모델임. 이때 충분한 데이터가 없으면 과적합이 발생하는 등 테스트 데이터에 대한 성능이 좋지 않음. 우리가 사용할 예제에서는 데이터셋을 상당히 제한하여 사용하고 있기 때문에 성능은 좋지 않다는 것을 미리 언급함. 성능이 좋은 결과를 원한다면 충분한 데이터셋을 확보하고 테스트를 진행하면 됨. 예를 들어 캐글에서 내려받은 모든 이미지를 사용하는 것뿐만 아니라 전처리 부분에서 데이터를 많이 확장(RandomRotation, RandomHorizontalFlip 등을 이용)시켜 예제를 진행해야 함.

`torch.utils.data.Dataset` 을 상속받아 커스텀 데이터셋(custom dataset)을 정의함.

`torch.utils.data.Dataset` 클래스를 상속받아 커스텀 데이터셋을 만들어 보겠음.

코드 6-25 커스텀 데이터셋 정의

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):
        self.file_list = file_list ----- 이미지 데이터가 위치한 파일 경로
        self.transform = transform ----- 이미지 데이터 전처리
        self.phase = phase ----- self.phase는 ImageTransform()에서 정의한 'train'과 'val'을 의미

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx] ----- 이미지 데이터의 인덱스를 가져오기
        img = Image.open(img_path)
        img_transformed = self.transform(img, self.phase)

        label = img_path.split('/')[-1].split('.')[0] ----- 레이블 값을 가져오기
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0

        return img_transformed, label ----- 전처리가 적용된 이미지와 레이블 반환
```

전처리에서 필요한 평균(mean), 표준편차(std) 등에 대한 변수 값을 정의함.

코드 6-26 변수에 대한 값 정의

```
size = 256 ----- AlexNet은 깊이가 깊은 네트워크를 사용하므로 이미지 크기가 256이 아니면
                    풀링층 때문에 크기가 계속 줄어들어 오류가 발생할 수 있습니다.
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

훈련과 검증 용도의 데이터셋을 정의함. 앞에서 정의한 DogvsCatDataset() 클래스에 훈련, 검증, 테스트 데이터를 적용하되 전처리도 함께 적용하도록 함.

코드 6-27 훈련, 검증, 테스트 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size,
                                                                                   mean, std), phase='train')
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size,
                                                                                mean, std), phase='val')
```

```
test_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size,
                                                                              mean, std), phase='val')

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])
```

다음은 훈련 데이터셋의 크기 및 레이블에 대한 출력 결과임.

```
torch.Size([3, 256, 256])
1
```

훈련 데이터셋의 크기는 (3, 256, 256)을 보여 줌. 이것에 대한 값은 (채널, 너비, 높이)를 의미함.

이제 데이터셋을 데이터로더로 전달하여 메모리로 불러올 준비를 함.

코드 6-28 데이터셋을 메모리로 불러옴

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

다음은 데이터로더에서 불러온 훈련 이미지에 대한 크기와 레이블을 출력한 결과임.

```
torch.Size([32, 3, 256, 256])
tensor([0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0,
1, 1, 1, 0, 0, 1])
```

데이터 관련 준비는 완료되었음. 이제 AlexNet 모델에 대한 네트워크를 정의함. AlexNet 모델을 사용하기 위한 네트워크는 사전 훈련된 네트워크와 유사하게 정의했음. 합성곱(Conv2d)+(활성화 함수(ReLU))+풀링(MaxPool2d)이 다섯 번 반복된 후 두 개의 완전연결층과 출력층으로 구성된 네트워크임.

```
class AlexNet(nn.Module):
    def __init__(self) -> None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True), ----- ①
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6)) ----- ②
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256*6*6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, 2),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

① ReLU 활성화 함수에서 inplace 의미에 대해 대략 살펴보았음. inplace 연산이란 연산에 대한 결괏값을 새로운 변수에 저장하는 것이 아닌 기존 데이터를 대체하는 것을 의미함. 즉, 기존 값을 연산 결괏값으로 대체함으로써 기존 값들을 무시하겠다는 의미임.

② nn.AdaptiveAvgPool2d는 nn.AvgPool2d처럼 풀링을 위해 사용함. AvgPool2d에서는 풀링에 대한 커널 및 스트라이드 크기를 정의해야 동작함. 예를 들어 nn.AvgPool2d((3, 2), stride=(2, 1))처럼 지정하면 그 결과는 5×5 텐서를 3×3 텐서로, 7×7 텐서를 4×4 텐서로 줄이는 효과를 얻을 수 있음. 조금 더 정확히 표현하자면, (N, C, H_{in}, W_{in}) 크기의 입력을 (N, C, H_{out}, W_{out}) 크기로 출력하는 것이 AvgPool2d임. 이때 H_{out}, W_{out}은 다음과 같은 공식에 의해 계산됨.

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times \text{padding}[0] - \text{kernel_size}[0]}{\text{stride}[0]} + 1 \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in} + 2 \times \text{padding}[1] - \text{kernel_size}[1]}{\text{stride}[1]} + 1 \right\rceil$$

반면에 AdaptiveAvgPool2d는 풀링 작업이 끝날 때 필요한 출력 크기를 정의함. 즉, nn.AvgPool2d에서는 커널 크기, 스트라이드, 패딩을 지정했다면 nn.AdaptiveAvgPool2d는 출력에 대한 크기만 지정함. 앞의 수식에서 H_{out} , W_{out} 값이 정해졌기 때문에 커널 크기, 스트라이드, 패딩 값을 구할 수 있음. 따라서 AdaptiveAvgPool2d를 사용할 경우 출력 크기에 대한 조정이 상당히 쉬워짐. 참고로 AdaptiveAvgPool2d는 입력 크기에 변동이 있고 CNN 위쪽에 완전연결층을 사용하는 경우 유용함.

앞서 생성한 모델의 네트워크 클래스(AlexNet)를 호출하여 model이라는 객체를 생성함.

코드 6-30 model 객체 생성

```
model = AlexNet()
model.to(device)
```

코드를 실행하면 AlexNet 모델에 대한 네트워크 구조를 보여 줌.

```
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
```

```

(7): ReLU(inplace=True)
(8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(9): ReLU(inplace=True)
(10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(11): ReLU(inplace=True)
(12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
)
(avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
(classifier): Sequential(
  (0): Dropout(p=0.5, inplace=False)
  (1): Linear(in_features=9216, out_features=4096, bias=True)
  (2): ReLU(inplace=True)
  (3): Dropout(p=0.5, inplace=False)
  (4): Linear(in_features=4096, out_features=512, bias=True)
  (5): ReLU(inplace=True)
  (6): Linear(in_features=512, out_features=2, bias=True)
)
)

```

학습에서 사용될 옵티마이저와 손실 함수를 정의함.

코드 6-31 옵티마이저 및 손실 함수 정의

```

optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()

```

torchsummary를 이용하여 모델의 네트워크를 다시 살펴보겠음. 모델의 네트워크를 파라미터와 함께 보여 주고 있기 때문에 내부적으로 어떤 일들이 이루어지는지 예측하면서 살펴볼 수 있음.

코드 6-32 모델 네트워크 구조 확인

```

from torchsummary import summary
summary(model, input_size=(3, 256, 256))

```

다음은 torchsummary를 이용한 네트워크 구조를 출력한 결과임.

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 63, 63]	23,296
ReLU-2	[-1, 64, 63, 63]	0

MaxPool2d-3	[-1, 64, 31, 31]	0
Conv2d-4	[-1, 192, 31, 31]	307,392
ReLU-5	[-1, 192, 31, 31]	0
MaxPool2d-6	[-1, 192, 15, 15]	0
Conv2d-7	[-1, 384, 15, 15]	663,936
ReLU-8	[-1, 384, 15, 15]	0
Conv2d-9	[-1, 256, 15, 15]	884,992
ReLU-10	[-1, 256, 15, 15]	0
Conv2d-11	[-1, 256, 15, 15]	590,080
ReLU-12	[-1, 256, 15, 15]	0
MaxPool2d-13	[-1, 256, 7, 7]	0
AdaptiveAvgPool2d-14	[-1, 256, 6, 6]	0
Dropout-15	[-1, 9216]	0
Linear-16	[-1, 4096]	37,752,832
ReLU-17	[-1, 4096]	0
Dropout-18	[-1, 4096]	0
Linear-19	[-1, 512]	2,097,664
ReLU-20	[-1, 512]	0
Linear-21	[-1, 2]	1,026

=====

Total params: 42,321,218

Trainable params: 42,321,218

Non-trainable params: 0

Input size (MB): 0.75

Forward/backward pass size (MB): 10.90

Params size (MB): 161.44

Estimated Total Size (MB): 173.10

어떤가? 모델의 네트워크가 한눈에 들어오지 않는가?

그럼 이제 모델 학습을 진행할 함수를 정의해 보겠음. 이 부분 역시 LeNet에서 사용했던 코드와 같기 때문에 설명은 생략함.

코드 6-33 모델 학습 함수 정의

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):

    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch+1, num_epoch))

        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]):
                inputs = inputs.to(device)
                labels = labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                if phase == 'train':
                    loss.backward()
                    optimizer.step()

                epoch_loss += loss.item() * inputs.size(0)
                epoch_corrects += torch.sum(preds == labels.data)

            epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
            epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)

            print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(
        time_elapsed // 60, time_elapsed % 60))
    return model
```

데이터셋 준비, 모델 학습을 위한 함수가 완료되었음. 훈련을 위해 정의된 `train_model()` 함수를 호출하여 모델을 학습시켜 봄. 이때 에포크는 10으로 함. 이 역시 예제를 진행할 PC(노트북)의 성능 및 학습 시간을 고려하여 최소한의 에포크로 지정했음.

코드 6-34 모델 학습

```
num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)
```

다음은 모델 학습에 대한 결과임. 역시 훈련과 검증 데이터셋이 적기 때문에 학습 결과가 좋지 않음. 데이터셋을 증가시키면 결과가 좋아질 수 있으므로 PC(노트북)의 성능이 좋다면 데이터셋을 증가시켜 학습해 보기 바람.

Epoch 1/10

```
-----
100% ██████████ 13/13 [00:31<00:00, 2.15s/it]
train Loss: 0.6929 Acc: 0.4950
100% ██████████ 3/3 [00:03<00:00, 1.15s/it]
val Loss: 0.6926 Acc: 0.5109
```

Epoch 2/10

```
-----
... 중간 생략 ...
-----
100% ██████████ 13/13 [00:37<00:00, 2.40s/it]
train Loss: 0.6925 Acc: 0.5025
100% ██████████ 3/3 [00:03<00:00, 1.11s/it]
val Loss: 0.6924 Acc: 0.5109
```

Epoch 10/10

```
-----
100% ██████████ 13/13 [00:38<00:00, 2.29s/it]
train Loss: 0.6924 Acc: 0.5025
100% ██████████ 3/3 [00:03<00:00, 1.05s/it]
val Loss: 0.6924 Acc: 0.5109
Training complete in 6m 9s
```

모델 학습이 끝났으므로 모델을 이용하여 테스트 데이터셋을 예측해 보겠음.

코드 6-35 모델을 이용한 예측

```

import pandas as pd
id_list = []
pred_list = []
_id = 0
with torch.no_grad():
    for test_path in tqdm(test_images_filepaths): ----- 테스트 이미지 데이터 이용
        img = Image.open(test_path)
        _id = test_path.split('/')[1].split('.')[1] ----- 이미지 데이터의 번호 가져오기(예를 들어
        transform = ImageTransform(size, mean, std)          dog.113.jpg라는 이미지 이름에서 113 가져오기)

        img = transform(img, phase='val') ----- 테스트 데이터에 검증용 전처리 적용
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs, dim=1)[: , 1].tolist()

        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
}) ----- 데이터 프레임에 이미지의 id(번호)와 레이블 저장
res.to_csv('../chap06/data/alexnet.csv', index=False) ----- alexnet.csv 파일에 저장

```

다음은 모델 예측에 대한 실행 결과임. 예측 결과를 CSV 파일로 저장하는 함수이기 때문에 결과는 큰 의미 없으며, 단순히 처리가 완료된 결과를 보여 줌.

```

100%
10/10 [00:00<00:00, 16.72it/s]

```

데이터 프레임에 대한 결과를 확인해 봄.

코드 6-36 데이터 프레임 결과 확인

```
res.head(10)
```

다음은 데이터 프레임에 대한 출력 결과임.

	id	label
0	207	0.493234
1	99	0.494147
2	116	0.493472
3	42	0.493139
4	162	0.493536
5	194	0.493116
6	115	0.493754
7	186	0.493568
8	39	0.492973
9	190	0.493639

label 값이 0.5보다 크면 개, 0.5보다 작으면 고양이로 예측했음을 의미함.

다음은 예측 결과를 시각적으로 표현하기 위한 함수를 정의함.

코드 6-37 예측 결과를 시각적으로 표현하기 위한 함수 정의

```
class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]
        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
```

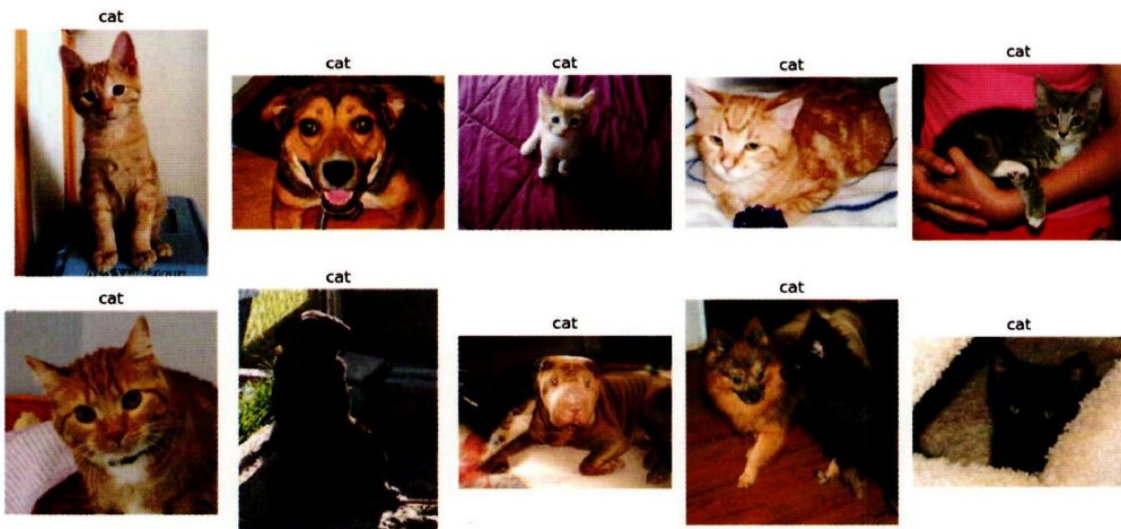
마지막으로 함수를 호출하여 테스트 데이터셋에 대한 예측 결과를 이미지와 함께 보여 줌.

코드 6-38 예측 결과에 대해 이미지와 함께 출력

```
display_image_grid(test_images_filepaths)
```

그림 6-12는 모델이 분류한 예측 결과를 보여 줌.

♥ 그림 6-12 테스트 데이터셋에 대한 AlexNet 모델의 예측 결과



역시 예측 결과가 좋지 않음. 앞에서 언급했지만 데이터셋이 많으면 성능이 좋아질 수 있음. 책의 예제들은 실습할 PC(노트북)의 성능을 고려하여 데이터셋을 제한시켰음. 성능은 파라미터 튜닝을 통해서도 향상시킬 수 있음. 자세한 내용은 '8장 성능 최적화'에서 다룸.

6.1.3 VGGNet

VGGNet은 카렌 시모니안(Karen Simonyan)과 앤드류 지서만(Andrew Zisserman)이 2015 ICLR에 게재한 "Very deep convolutional networks for large-scale image recognition" 논문에서 처음 발표했음. VGGNet은 합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하려고 탄생했음. 즉, 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 것이 VGG임. VGG 연구 팀은 깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3×3 으로 고정했음.

네트워크 계층의 총 개수에 따라 여러 유형의 VGGNet(VGG16, VGG19 등)이 있으며, 이 중 VGG16 네트워크의 구조적 세부 사항은 다음 그림과 같음.

VGG16에는 파라미터가 총 1억 3300만 개 있음. 여기에서 주목할 점은 모든 합성곱 커널의 크기는 3×3 , 최대 풀링 커널의 크기는 2×2 이며, 스트라이드는 2라는 것임. 결과적으로 64개의 224×224 특성 맵 ($224 \times 224 \times 64$)들이 생성됨. 또한, 마지막 16번째 계층을 제외하고는 모두 ReLU 활성화 함수가 적용됨.

♥ 그림 6-13 VGG16 구조

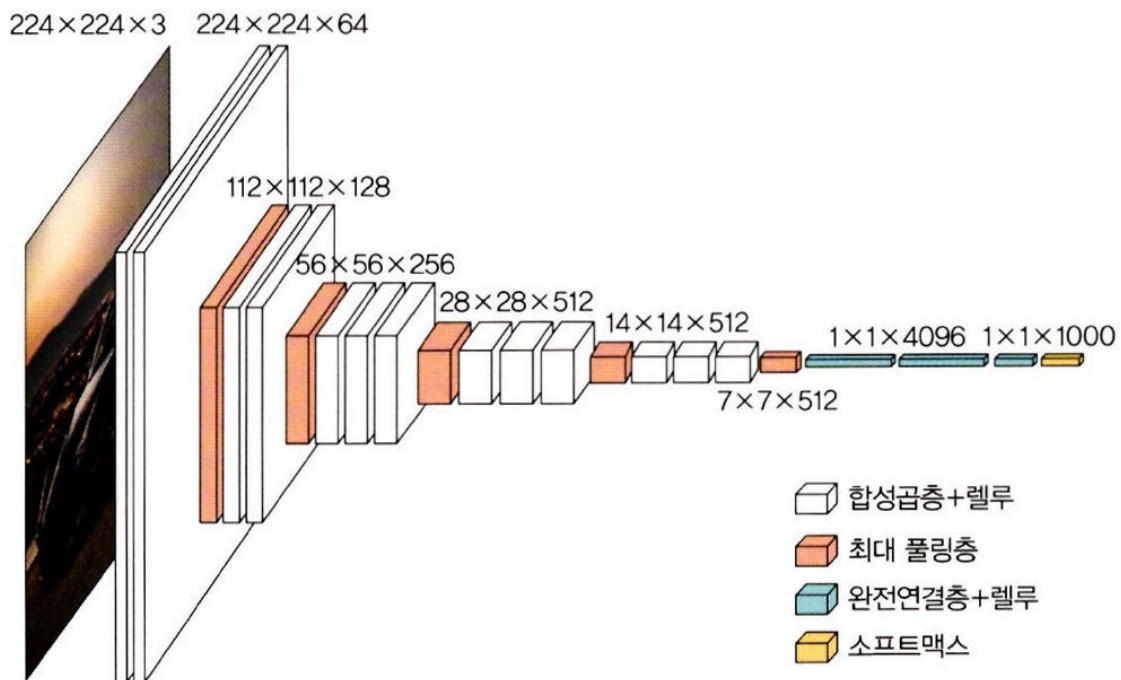


표 6-3 VGG16 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
최대 풀링층	128	56×56	2×2	2	–
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	–
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	–
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	–
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	1000	–	–	소프트맥스 (softmax)

이번 예제는 VGGNet 중에서 가장 간단한 VGG11을 파이토치로 구현함. VGG16이나 VGG19를 구현해 보고 싶다면, VGG11에서 더 깊게 쌓아 올리면 됨.

가장 처음으로 할 일은 필요한 라이브러리를 호출하는 것임.

코드 6-39 필요한 라이브러리 호출

```
import copy ----- ①
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as Datasets

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

① 객체 복사를 위해 사용함. 객체 복사는 크게 얇은 복사(shallow copy)와 깊은 복사(deep copy)로 나뉨. 예제를 통해 살펴보겠음.

먼저 단순한 객체 복사는 다음과 같음.

```
original = [1, 2, 3] ----- original이라는 변수에 [1, 2, 3] 저장
copy_o = original ----- copy_o에 original 복사(shallow copy)
print(copy_o)

copy_o[2] = 10 ----- copy_o의 3을 10으로 값을 변경
print(copy_o)
print(original)
```

original 값을 복사한 copy_o에서 3 값을 10으로 바꾼 결과는 다음과 같음.

```
[1, 2, 3]
[1, 2, 10]
[1, 2, 10]
```

copy_o뿐만 아니라 원래 값인 original의 3도 10으로 바뀌었음.

이번에는 얇은 복사에 대해 알아보겠음. 얇은 복사는 copy.copy()를 이용함.

```
import copy

original = [[1, 2], 3]
copy_o = copy.copy(original) ----- original 값을 copy_o에 얇은 복사(copy.copy())
print(copy_o)
copy_o[0] = 100 ----- copy_o의 [1, 2] 값을 100으로 변경
print(copy_o)
print(original)

append = copy.copy(original)
append[0].append(4) ----- 첫 번째 리스트([1, 2])에 4를 추가
print(append)
print(original)
```

얕은 복사(copy.copy())에 대한 결과는 다음과 같음.

```
[[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2, 4], 3]
```

copy_o에서 [1, 2] 값을 100으로 변경했더니 copy_o만 바뀌었음. 또한, [1, 2]에 4를 추가했더니 original과 copy_o 모두 반영되었음.

그렇다면 깊은 복사는 어떨까? 깊은 복사는 copy.deepcopy()를 이용함.

```
import copy

original = [[1, 2], 3]
copy_o = copy.deepcopy(original) ----- original 값을 copy_o에 깊은 복사(copy.deepcopy())
print(copy_o)
copy_o[0] = 100 ----- copy_o의 [1, 2] 값을 100으로 변경
print(copy_o)
print(original)

append = copy.deepcopy(original)
append[0].append(4) ----- 첫 번째 리스트([1, 2])에 4를 추가
print(append)
print(original)
```

깊은 복사를 실행한 결과는 다음과 같음.

```

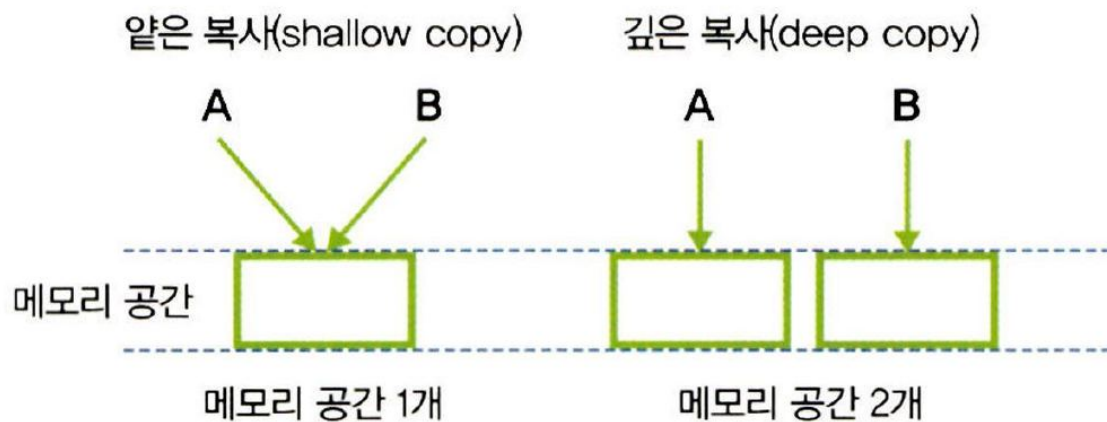
[[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2], 3]

```

copy_o에서 [1, 2] 값을 100으로 변경했더니 copy_o만 바뀐 것은 동일함. 또한, [1, 2]에 4를 추가했더니 copy_o는 변경되었지만, original은 그대로인 것을 확인할 수 있음.

단순 복사는 완전히 동일하게 복사되지만, 얇은 복사와 깊은 복사는 다음과 같은 차이가 있음.

▼ 그림 6-14 얇은 복사와 깊은 복사



데이터셋과 관련된 준비들이 완료되었으니 VGG 모델의 네트워크를 정의해 보겠습니다.

코드 6-40 VGG 모델 정의

```
class VGG(nn.Module):
    def __init__(self, features, output_dim):
        super().__init__()
        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d(7)
        self.classifier = nn.Sequential(
            nn.Linear(512*7*7, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, output_dim)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        h = x.view(x.shape[0], -1)
        x = self.classifier(h)
        return x, h
```

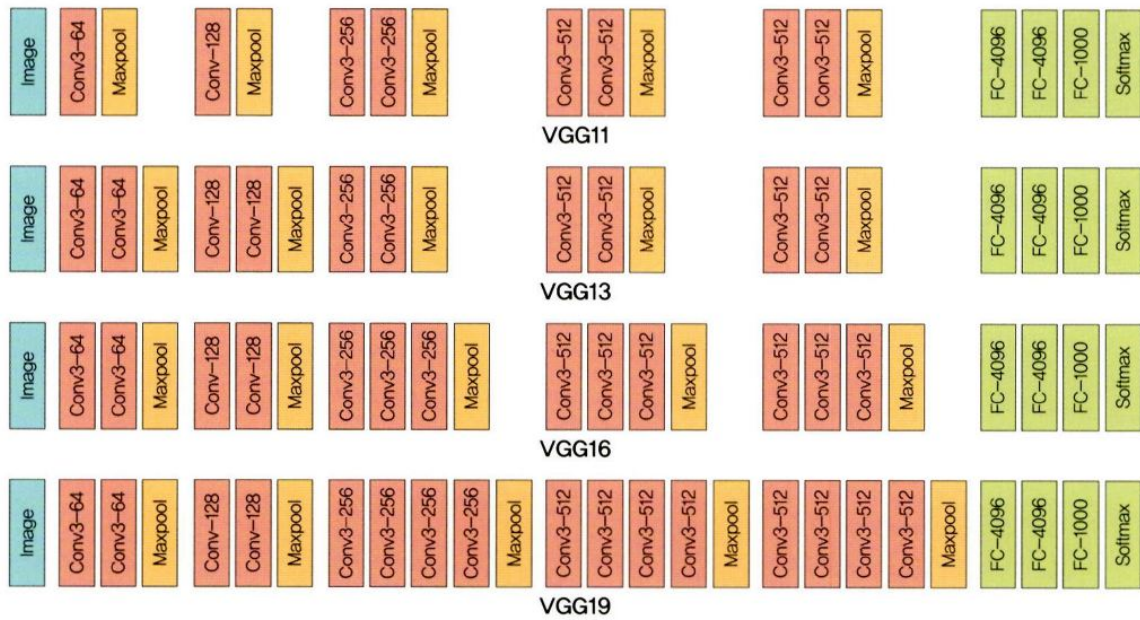
이번 코드는 VGG11, VGG13, VGG16, VGG19 모델의 계층을 정리한 것임. 숫자(output channel, 출력 채널)는 Conv2d를 수행하라는 의미이며, 출력 채널(output channel)이 다음 계층의 입력 채널(input channel)이 됨. 또한, M은 최대 풀링(max pooling)을 수행하라는 의미임.

코드 6-41 모델 유형 정의

```
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M']
vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 'M']
```

VGG11, VGG13, VGG16, VGG19에 대한 네트워크를 그림으로 정리하면 다음과 같음.

▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



VGG 모델에서 네트워크를 정의함. 네트워크는 코드 6-41에서 정의된 vgg11_config를 사용함.

코드 6-42 VGG 계층 정의

```
def get_vgg_layers(config, batch_norm):
    layers = []
    in_channels = 3

    for c in config: ----- vgg11_config 값들을 가져옵니다.
        assert c == 'M' or isinstance(c, int) ----- ①
        if c == 'M': ----- 불러온 값이 'M'이면 최대 풀링(MaxPool2d)을 적용
            layers += [nn.MaxPool2d(kernel_size = 2)]
        else: ----- 불러온 값이 숫자이면 합성곱(Conv2d) 적용
            conv2d = nn.Conv2d(in_channels, c, kernel_size=3, padding=1)
            if batch_norm: ----- 배치 정규화(batch normalization)를 적용할지에 대한 코드
                layers += [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace=True)] -----
            else: ----- 배치 정규화가 적용될 경우 배치 정규화+ReLU 적용
                layers += [conv2d, nn.ReLU(inplace=True)] -----
            in_channels = c ----- 배치 정규화가 적용되지 않을 경우 ReLU만 적용

    return nn.Sequential(*layers) ----- 네트워크의 모든 계층을 반환
```

① 조건문을 정의함.

assert c == 'M' or isinstance(c, int)

Ⓐ

Ⓑ

㉑ 가정 설정문이라고 불리는 `assert`는 뒤의 조건이 `True`가 아니면 에러를 발생시킴. 따라서 `c == 'M'`이 아니면 오류가 발생함. 예를 들어 다음과 같음.

```
a = 1
assert a == 11
```

조건이 `True`가 아니기 때문에 다음과 같은 오류가 발생함.

```
-----
AssertionError                                Traceback (most recent call last)
<ipython-input-1-b211624433f8> in <module>
      1 a = 1
----> 2 assert a == 11

AssertionError:
```

㉒ `isinstance`는 주어진 조건이 `True`인지 판단함. 예를 들어 다음과 같이 사용함.

```
print(isinstance(1, int)) ----- 1이 integer인지 판단
print(isinstance(1.2, int)) ----- 1.2가 integer인지 판단
print(isinstance('deep learning', str)) ----- deep learning이 string인지 판단
```

다음은 `isinstance`로 조건이 `True/False`인지 확인한 결과임.

```
True
False
True
```

따라서 `assert c == 'M' or isinstance(c, int)` 의미는 `c`가 `'M'`이 아니거나 `int`가 아니라면 오류가 발생함.

`get_vgg_layers()` 함수를 호출하여 모델의 계층을 생성함. 이때 배치 정규화(batch normalization)에 대한 계층도 추가함.

코드 6-43 모델 계층 생성

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True) ----- ①
```

① batch_norm(Batch Normalization)은 데이터의 평균을 0으로, 표준편차를 1로 분포시키는 것임. 각 계층에서 입력 데이터의 분포는 앞 계층에서 업데이트된 가중치에 따라 변함. 즉, 각 계층마다 변화되는 분포는 학습 속도를 늦출 뿐만 아니라 학습도 어렵게 함. 따라서 각 계층의 입력에 대한 분산을 평균 0, 표준편차 1로 분포시키는 것이 batch_norm(배치 정규화)임. 배치 정규화는 8장에서 자세히 다룸.

생성한 계층을 확인해 보겠음.

코드 6-44 VGG11 계층 확인

```
print(vgg11_layers)
```

다음은 VGG11 계층을 출력한 결과임.

```
Sequential(
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
  (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (6): ReLU(inplace=True)
  (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (10): ReLU(inplace=True)
  (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (13): ReLU(inplace=True)
  (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (17): ReLU(inplace=True)
  (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (20): ReLU(inplace=True)
  (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (24): ReLU(inplace=True)
  (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```

(27): ReLU(inplace=True)
(28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)

```

VGG11에 대한 계층을 확인했음. 확인 결과 배치 정규화(BatchNorm2d)가 추가된 것을 확인할 수 있음.

이제 모델 전체에 대한 네트워크를 확인해 봄. 네트워크는 vgg11_layers와 VGG()에서 정의했던 완전연결층과 출력층이 포함된 구성일 것임.

코드 6-45 VGG11 전체에 대한 네트워크

```

OUTPUT_DIM = 2 ----- 개와 고양이 두 개의 클래스 사용
model = VGG(vgg11_layers, OUTPUT_DIM)
print(model)

```

다음은 VGG11 전체에 대한 네트워크를 출력한 결과임.

```

VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
    (13): ReLU(inplace=True)
    (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_

```

```

stats=True)
    (20): ReLU(inplace=True)
    (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
    (24): ReLU(inplace=True)
    (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
    (27): ReLU(inplace=True)
    (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
(avgpool): AdaptiveAvgPool2d(output_size=7)
(classifier): Sequential(
  (0): Linear(in_features=25088, out_features=4096, bias=True)
  (1): ReLU(inplace=True)
  (2): Dropout(p=0.5, inplace=False)
  (3): Linear(in_features=4096, out_features=4096, bias=True)
  (4): ReLU(inplace=True)
  (5): Dropout(p=0.5, inplace=False)
  (6): Linear(in_features=4096, out_features=2, bias=True)
)
)

```

어떤가? 앞에서 정의했던 `vgg11_layers`, 완전연결층과 출력층(`(classifier): Sequential()`) 부분을 합친 것을 확인할 수 있음.

VGG11 모델을 사용하는 것이 어렵게 느껴지는가? 그렇다면 VGG11을 더 쉽게 사용할 수 있는 방법이 있음. 사실 VGG 모델은 사전 훈련된 모델임. 이미 누군가가 대용량의 이미지 데이터로 학습을 시켰으며, 최상의 상태로 튜닝을 거쳐 모든 사람이 사용할 수 있도록 공유한 사전 훈련된 모델임. 따라서 다음과 같이 코드 한두 줄로 쉽게 가져다 사용할 수 있음.

코드 6-46 VGG11 사전 훈련된 모델 사용

```

import torchvision.models as models
pretrained_model = models.vgg11_bn(pretrained=True) ----- ①
print(pretrained_model)

```

① 배치 정규화가 적용된 사전 훈련된 VGG11 모델을 사용하기 위해서는 다음과 같은 파라미터를 사용함.

```

pretrained_model = models.vgg11_bn(pretrained=True)
                    (a)                (b)

```

ⓐ `vgg11_bn`은 VGG11 기본 모델에 배치 정규화가 적용된 모델을 사용하겠다는 의미임.

⑥ pretrained를 True로 설정하면 사전 훈련된 모델을 사용(미리 학습된 파라미터 값들을 사용)하겠다는 의미임.

다음은 사전 훈련된 VGG11 네트워크를 출력한 결과임.

```
VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): ReLU(inplace=True)
    (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (20): ReLU(inplace=True)
    (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (24): ReLU(inplace=True)
    (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```

(27): ReLU(inplace=True)
(28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
(avgpool): AdaptiveAvgPool2d(output_size=(7, 7))
(classifier): Sequential(
  (0): Linear(in_features=25088, out_features=4096, bias=True)
  (1): ReLU(inplace=True)
  (2): Dropout(p=0.5, inplace=False)
  (3): Linear(in_features=4096, out_features=4096, bias=True)
  (4): ReLU(inplace=True)
  (5): Dropout(p=0.5, inplace=False)
  (6): Linear(in_features=4096, out_features=1000, bias=True)
)
)

```

어떤가? 앞에서 수십 줄 넘게 구현한 코드가 의미 없다고 느껴지는가? 물론 향후에는 연습을 하든 연구를 하든 사전 훈련된 모델을 호출하여 사용하게 될 것임. 하지만 책에서는 VGG 모델의 네트워크를 어떻게 구현하고 사용하는지 방법을 알려 주고자 고생스럽지만 네트워크를 정의했던 것임. 이러한 수고를 통해 우리는 다음과 같이 My_Vgg 같은 새로운 config를 정의하여 사용할 수 있음.

```

vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512,
                'M']
vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M',
                512, 512, 512, 'M']
vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512,
                512, 'M', 512, 512, 512, 512, 'M']
My_Vgg = [64, 64, 64, 'M', 128, 128, 128, 'M', 256, 256, 256, 'M']

```

이제 이미지에 대한 전처리 부분을 정의함.

코드 6-47 이미지 데이터 전처리

```

train_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomRotation(5),
    transforms.RandomHorizontalFlip(0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
                                                            0.224, 0.225])) ----- ①

test_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
                                                            0.224, 0.225]))

```

① 데이터 전처리에 사용되는 transforms.Compose 파라미터 중 새롭게 등장하는 것들만 알아보겠습니다.

```
transforms.Compose([transforms.Resize((256, 256)), transforms.RandomRotation(5),])
```

(a) (b)

① transforms.Resize: 이미지를 주어진 크기로 재조정함. 즉, 256×256 크기로 이미지를 조정함.

② transforms.RandomRotation: 5도 이하로 이미지를 회전시킴.

ImageFolder를 이용하여 모델 학습에 필요한 데이터셋을 불러옴.

코드 6-48 ImageFolder를 이용하여 데이터셋 불러오기

```
train_path = '../chap06/data/catanddog/train' ----- 훈련 데이터셋이 위치한 경로
test_path = '../chap06/data/catanddog/test' ----- 테스트 데이터셋이 위치한 경로

train_dataset = torchvision.datasets.ImageFolder(
    train_path,
    transform=train_transforms
) ----- ①

test_dataset = torchvision.datasets.ImageFolder(
    test_path,
    transform=test_transforms
)

print(len(train_dataset)), print(len(test_dataset))
```

① ImageFolder에서 사용하는 파라미터는 5장에서 살펴보았음. 따라서 여기에서는 ImageFolder의 파라미터가 아닌 언제 사용하면 좋을지에 대해 설명함. ImageFolder는 계층적인 폴더 구조를 가지고 있는 데이터셋을 불러올 때 사용함. 예를 들어 훈련 데이터셋이 위치한 '../chap06/data/catanddog/train'에는 다음과 같은 폴더들이 있음.

▼ 그림 6-16 '../chap06/data/catanddog/train' 위치의 하위 폴더

☐ 이름	유형
 Cat	파일 폴더
 Dog	파일 폴더

이와 같이 train 폴더 하위에 Cat, Dog 폴더가 계층적으로 위치해 있고 이러한 구조에서 데이터셋을 불러오고 싶을 때 ImageFolder를 사용함.

다음은 훈련과 테스트 데이터로 사용될 이미지 수임. 훈련 용도의 데이터는 529개가 사용되며, 테스트 용도의 데이터는 12개가 사용됨.

529
12
(None, None)

훈련 데이터를 훈련과 검증 데이터로 분할함.

코드 6-49 훈련과 검증 데이터 분할

```
VALID_RATIO = 0.9
n_train_examples = int(len(train_dataset) * VALID_RATIO)
n_valid_examples = len(train_dataset) - n_train_examples

train_data, valid_data = data.random_split(train_dataset,
                                           [n_train_examples, n_valid_examples])
```

----- ①

----- 전체 훈련 데이터 중 90%를
훈련 데이터셋으로 사용

----- 전체 훈련 데이터 중 10%를
검증 데이터셋으로 사용

① 파이썬의 `random_split()`은 훈련과 검증 데이터셋을 나누는 용도로 사용하며 다음과 같은 파라미터를 사용함. 데이터가 데이터로더로 넘어간 이후에는 분리가 불가능하므로 데이터셋(dataset) 단계에서 진행해야 함.

```
train_data, valid_data = data.random_split(train_dataset,
                                           (a)
                                           [n_train_examples, n_valid_examples])
                                           (b)
```

① 첫 번째 파라미터: 분할에 사용될 데이터셋

② 두 번째 파라미터: 훈련과 검증 데이터셋의 크기 지정. [훈련 데이터셋 크기, 검증 데이터셋 크기]

앞에서 훈련 데이터셋으로부터 분리된 `valid_data`를 `valid_data`라는 변수에 복사한 후 'test_transforms'로 전처리를 적용함.

코드 6-50 검증 데이터 전처리

```
valid_data = copy.deepcopy(valid_data)
valid_data.dataset.transform = test_transforms
```

데이터가 훈련, 검증, 테스트로 분류되었기 때문에 각각 몇 개의 이미지 데이터가 있는지 확인함.

코드 6-51 훈련, 검증, 테스트 데이터셋 수 확인

```
print(f'Number of training examples: {len(train_data)}')
print(f'Number of validation examples: {len(valid_data)}')
print(f'Number of testing examples: {len(test_dataset)}')
```

다음은 훈련, 검증, 테스트로 분류된 데이터들의 수를 출력한 결과임.

```
Number of training examples: 476
Number of validation examples: 53
Number of testing examples: 12
```

훈련용은 476개, 검증용은 53개, 테스트용은 12개의 데이터셋으로 구성된 것을 확인할 수 있음.

데이터로더를 이용하여 데이터셋의 데이터를 메모리로 가져옴. 데이터를 가져올 때는 배치 크기만큼 나누어서 가져옴.

코드 6-52 메모리로 데이터 불러오기

```
BATCH_SIZE = 128
train_iterator = data.DataLoader(train_data,
                                shuffle=True,
                                batch_size=BATCH_SIZE)
valid_iterator = data.DataLoader(valid_data,
                                batch_size=BATCH_SIZE)
test_iterator = data.DataLoader(test_dataset,
                                batch_size=BATCH_SIZE)
```

..... 훈련 데이터셋은 임의로 섞어서 가져옵니다.

모델 학습에서 사용할 옵티마이저와 손실 함수를 정의함.

코드 6-53 옵티마이저와 손실 함수 정의

```
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)
```

모델에 대한 정확도를 측정하기 위한 함수를 정의함.

코드 6-54 모델 정확도 측정 함수

```
def calculate_accuracy(y_pred, y):
    top_pred = y_pred.argmax(1, keepdim=True)
    correct = top_pred.eq(y.view_as(top_pred)).sum() ----- ①
    acc = correct.float() / y.shape[0]
    return acc
```

① 예측이 정답과 일치하는 경우 그 개수의 합을 correct 변수에 저장함.

correct = top_pred.eq(y.view_as(top_pred)).sum()

(a) (b) (c)

① eq는 equal의 약자로 서로 같은지를 비교하는 표현식임.

② view_as(other)는 other의 텐서 크기를 사용하겠다는 의미임. 즉, view_as(other)는 view(other.size())와 같은 의미임. 따라서 y.view_as(top_pred)는 y에 대한 텐서 크기를 top_pred의 텐서 크기로 변경하겠다는 의미임.

③ 합계를 구하는 것으로, 여기에서는 예측과 정답이 일치하는 것들의 개수를 합산하겠다는 의미임.

훈련 데이터셋을 이용할 모델 학습 함수를 정의함.

코드 6-55 모델 학습 함수 정의

```
def train(model, iterator, optimizer, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.train()

    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred, _ = model(x)
        loss = criterion(y_pred, y)
        acc = calculate_accuracy(y_pred, y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc += acc.item()
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

검증 및 테스트 데이터셋을 이용한 모델 성능을 측정하는 함수를 정의함.

코드 6-56 모델 성능 측정 함수

```
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred, _ = model(x)
            loss = criterion(y_pred, y)
            acc = calculate_accuracy(y_pred, y)
            epoch_loss += loss.item()
            epoch_acc += acc.item()
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

모델의 학습 시간(시작, 종료)을 측정하기 위한 함수를 정의함.

코드 6-57 학습 시간 측정 함수

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```

이제 모델을 학습시킴.

코드 6-58 모델 학습

```
EPOCHS = 5
best_valid_loss = float('inf')
for epoch in range(EPOCHS):
    start_time = time.monotonic()
    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    valid_loss, valid_acc = evaluate(model, valid_iterator, criterion, device)
    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), '../chap06/data/VGG-model.pt')
    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
    print(f'\t Valid. Loss: {valid_loss:.3f} | Valid. Acc: {valid_acc*100:.2f}%')
```

다음은 훈련과 검증 데이터셋을 이용한 모델 학습 결과임.

```
Epoch: 01 | Epoch Time: 8m 27s
    Train Loss: 0.691 | Train Acc: 54.48%
    Valid. Loss: 0.693 | Valid. Acc: 52.83%
Epoch: 02 | Epoch Time: 7m 40s
    Train Loss: 0.690 | Train Acc: 55.38%
    Valid. Loss: 0.693 | Valid. Acc: 52.83%
Epoch: 03 | Epoch Time: 7m 34s
    Train Loss: 0.699 | Train Acc: 50.73%
    Valid. Loss: 0.693 | Valid. Acc: 60.38%
Epoch: 04 | Epoch Time: 7m 33s
    Train Loss: 0.689 | Train Acc: 53.29%
    Valid. Loss: 0.693 | Valid. Acc: 47.17%
Epoch: 05 | Epoch Time: 7m 28s
    Train Loss: 0.700 | Train Acc: 51.52%
    Valid. Loss: 0.693 | Valid. Acc: 47.17%
```

역시 성능이 좋지 않음. 데이터셋으로 사용되는 이미지 수가 매우 적으며, 에포크도 매우 적게 설정되었음. 이들에 대한 숫자를 늘리면 성능이 좋아질 것임. 하지만 현재의 데이터셋에서 에포크만 늘린다고 성능이 좋아지는 않기 때문에 데이터도 함께 증가시켜 모델을 학습시켜야 함.

앞에서 저장했던 모델(`VGG-model.pt`)을 불러와서 테스트 데이터셋에 대한 성능을 측정함.

코드 6-59 테스트 데이터셋을 이용한 모델 성능 측정

```
model.load_state_dict(torch.load('../chap06/data/VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')
```

다음은 테스트 데이터셋을 이용한 모델 성능 측정 결과임.

Test Loss: 0.693 | Test Acc: 50.00%

역시 결과가 좋지 않음. 계속 이야기했듯이 데이터셋을 늘리면 성능이 좋아질 것임. 단 이미지 데이터가 늘어난 만큼 훈련 시간이 상당히 길어질 수 있음.

테스트 데이터셋을 이용한 모델의 예측 결과를 알아보기 위한 함수를 정의함.

코드 6-60 테스트 데이터셋을 이용한 모델의 예측 확인 함수

```
def get_predictions(model, iterator):
    model.eval()
    images = []
    labels = []
    probs = []

    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y_pred, _ = model(x)
            y_prob = F.softmax(y_pred, dim=-1)
            top_pred = y_prob.argmax(1, keepdim=True) ----- ①
            images.append(x.cpu())
            labels.append(y.cpu())
            probs.append(y_prob.cpu())

    images = torch.cat(images, dim=0) ----- ②
    labels = torch.cat(labels, dim=0)
    probs = torch.cat(probs, dim=0)
    return images, labels, probs
```

① `argmax`는 배열에서 가장 큰 값의 인덱스를 찾을 때 사용함.

`top_pred = y_prob.argmax(1, keepdim=True)`

(a) (b)

- ① 첫 번째 파라미터: 행(axis=0) 또는 열(axis=1)을 따라 가장 큰 값의 색인을 찾음. 따라서 1은 열을 따라 가장 큰 값의 색인을 찾겠다는 의미임.
- ② `keepdim`: `keepdim=True`의 경우 출력 텐서를 입력과 동일한 크기로 유지하겠다는 의미임.
- ② `torch.cat`은 텐서를 연결할 때 사용함. 다음 예제를 통해 사용 방법을 확인할 수 있음. 그 전에 차원에 대한 이해를 돕기 위해 다음 그림을 살펴봄.

♥ 그림 6-17 차원(dim)

	Col1	Col2	Col3
Row1			
Row2			
Row3			

axis=0(dim=0)

axis=1(dim=1)

dim=0은 행을 의미하며, dim=1은 열을 의미함. 이제 코드를 통해 `torch.cat`을 알아봄.

```
import torch
x = torch.Tensor([[1, 2, 3], [2, 3, 4]])
y = torch.Tensor([[4, 5, 6], [5, 6, 7]])

print(torch.cat([x], dim=0)) ----- 행을 기준(dim=0)으로 x를 이어 붙입니다.
print('-----')
print(torch.cat([x, y])) ----- 단순히 x와 y를 결합하라는 의미이며 출력 결과는 (4×3) 형태가 됩니다.
print('-----')
print(torch.cat([x, y], dim=0)) ----- 행을 기준(dim=0)으로 x와 y를 이어 붙이면 (4×3) 텐서가 됩니다.
print('-----')
print(torch.cat([x, y], dim=1)) ----- 열을 기준(dim=1)으로 x와 y를 이어 붙이면 (2×6) 텐서가 됩니다.
```

다음은 `torch.cat`을 실행한 결과임.

```
tensor([[1., 2., 3.],  
        [2., 3., 4.]])
```

```
tensor([[1., 2., 3.],  
        [2., 3., 4.],
```

```
        [4., 5., 6.],  
        [5., 6., 7.]])
```

```
tensor([[1., 2., 3.],  
        [2., 3., 4.],  
        [4., 5., 6.],  
        [5., 6., 7.]])
```

```
tensor([[1., 2., 3., 4., 5., 6.],  
        [2., 3., 4., 5., 6., 7.]])
```

앞에서 정의한 `get_predictions()` 함수의 반환값을 각각 `images`, `labels`, `probs`에 저장하여 모델이 정확하게 예측한 이미지를 추출함.

코드 6-61 예측 중에서 정확하게 예측한 것을 추출

```
images, labels, probs = get_predictions(model, test_iterator)
pred_labels = torch.argmax(probs, 1) ----- ①
corrects = torch.eq(labels, pred_labels) ----- 예측과 정답이 같은지 비교
correct_examples = []

for image, label, prob, correct in zip(images, labels, probs, corrects): ----- ②
    if correct:
        correct_examples.append((image, label, prob))

correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values) ----- ③
```

① `argmax`는 몇 번 설명했음. `max`는 최댓값을 반환하고, `argmax`는 최댓값을 갖는 인덱스를 반환함. 다음 예제를 통해 최댓값과 인덱스에 대해 알아봄. 먼저 임의의 텐서를 생성함.

```
x = torch.rand(5,3) ----- (5×3) 형태의 텐서 생성
print(x)
```

다음과 같이 (5×3) 형태의 임의의 텐서가 생성되었음.

```
tensor([[0.6501, 0.6177, 0.5627],
        [0.3163, 0.5619, 0.6364],
        [0.8501, 0.9622, 0.4792],
        [0.1859, 0.6510, 0.1824],
        [0.4304, 0.1170, 0.8049]])
```

생성된 텐서를 이용하여 `max`와 `argmax`의 반환되는 값을 알아봄.

```
print(torch.max(x)) ----- max의 반환값 알아보기
print('-----')
print(torch.argmax(x)) ----- argmax의 반환값 알아보기
```

다음은 `max`와 `argmax`에 대한 반환값임. 다음 결과와 같이 `max`는 텐서의 요소 중 가장 높은 값을 반환하지만, `argmax`는 그 인덱스를 반환함.

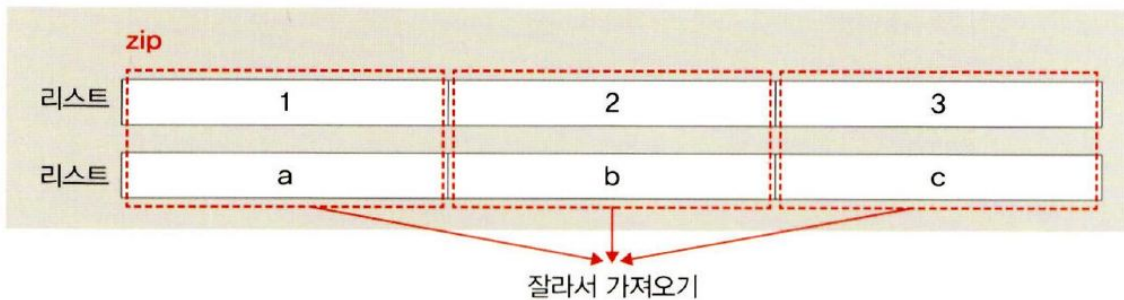
tensor(0.9622)

tensor(7)

② zip()은 여러 개의 리스트(혹은 튜플³)를 합쳐서 새로운 튜플 타입으로 반환함.

³ 튜플은 순서가 있는 데이터의 집합임. 순서가 있기 때문에 인덱스를 이용한 데이터 접근이 가능하지만, 한 번 생성되면 값을 변경할 수 없다는 단점이 있음.

▼ 그림 6-18 zip()의 원리



즉, 다음과 같이 사용함.

```
a = [1, 2, 3]
b = ['a', 'b', 'c']
```

```
for x, y in zip(a, b): ----- a, b가 리스트로 구성되었기 때문에 for 문을 통해 반복하여 값을 가져옵니다.
    print(x, y)
```

다음은 zip을 적용한 결과임.

```
1 a
2 b
3 c
```

③ 데이터를 정렬하기 위해 `sort()` 메서드를 사용하며, 파라미터는 다음과 같음.

```
correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)
```

(a) (b)

① `reverse`: 내림차순으로 정렬함. 예를 들어 다음과 같이 사용함.

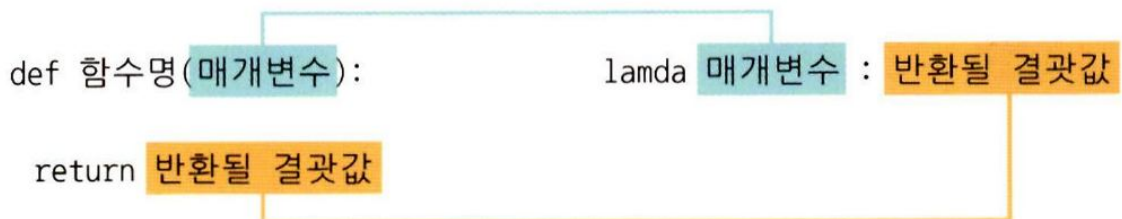
```
x = [1, 7, 3, 9] ----- 정렬되어 있지 않은 x라는 텐서 생성  
x.sort(reverse=True) ----- x에 내림차순 적용  
print('x:', x)
```

다음과 같이 x가 내림차순으로 정렬되었음.

```
x: [9, 7, 3, 1]
```

② `key`: 데이터를 정렬할 때 `key` 값을 가지고 정렬하며 기본값은 오름차순임. 또한, 여기서 사용되는 람다는 일종의 함수임. 일반적으로 함수는 `def 함수명()`처럼 사용하지만 람다 함수는 다음과 같이 함수명 없이도 사용 가능함.

▼ 그림 6-19 일반 함수와 람다 함수



람다 함수는 정의와 동시에 사용할 수 있지만 함수명이 없고, 저장된 변수가 없기 때문에 재사용은 불가능함. 예를 들어 사용 방법은 다음과 같음.

```
y = lambda x: x + 10  
y(1)
```

실행하면 다음과 같이 출력됨.

11

또한, `torch.max(x[2], dim=0)` 의미는 다음과 같음.

```
x = torch.randn([4, 4]) ----- (4×4) 크기를 갖는 임의의 텐서 생성
print(x)

max_elements, max_idx = torch.max(x, dim=0) ----- torch.max 값을 가지고 오되 dim=0(행을 기준)으로
print(max_elements)                               최댓값을 가져옵니다.
print(max_idx)
```

코드를 실행한 결과 행을 기준(dim=0)으로 최댓값을 가져온 것을 확인할 수 있음.

```
tensor([[ -0.3027,  -0.3185,   0.4117,   0.4915],
        [-3.1112,   1.3952,  -0.2522,  -0.5341],
        [-0.5381,  -0.2026,  -1.0494,  -1.2712],
        [-0.3689,  -1.8523,   2.0229,   0.5725]])
tensor([-0.3027,   1.3952,   2.0229,   0.5725])
tensor([0, 1, 3, 3])
```

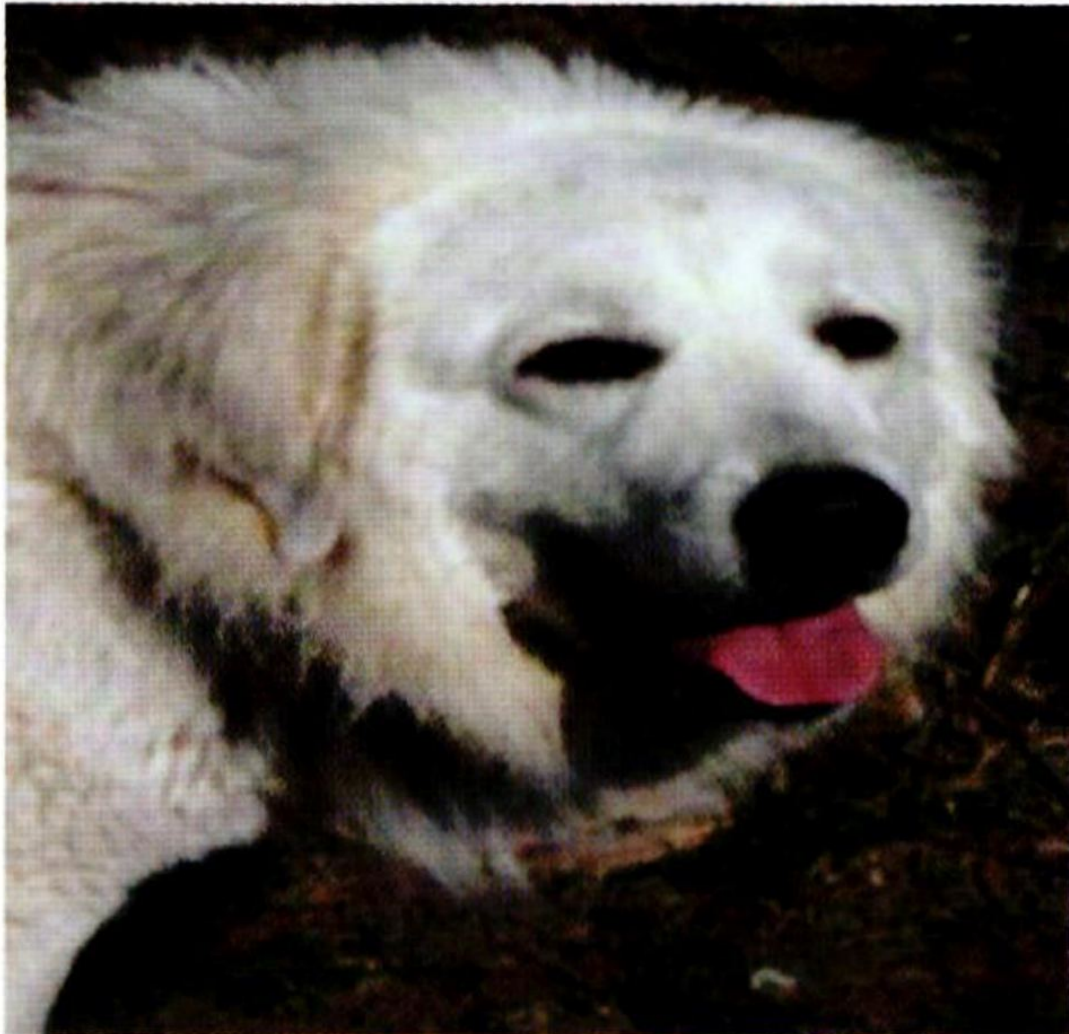
다음은 본래 이미지를 출력하기 위한 함수임. 이미 앞에서 이미지의 크기 및 전처리 과정 등을 거쳤기 때문에 이 상태에서 이미지를 출력하면 다음 그림과 같음.

♥ 그림 6-20 본래 이미지 색상이 왜곡됨



따라서 다음과 같이 본래 이미지를 출력하기 위해서는 코드 6-62와 같은 전처리가 필요함.

▼ 그림 6-21 본래 이미지



코드 6-62 이미지 출력을 위한 전처리

```
def normalize_image(image):  
    image_min = image.min()  
    image_max = image.max()  
    image.clamp_(min=image_min, max=image_max) ----- torch.clamp는 주어진 최소(min), 최대(max)의  
    image.add_(-image_min).div_(image_max-image_min+1e-5) ----- 범주에 이미지가 위치하도록 합니다.  
    return image ----- ①
```

① torch.add는 말 그대로 더하라는 메서드임. 또한, 단순히 torch.add가 아닌 torch.add_가 사용되고 있는데 이 둘 간의 차이는 다음과 같음.

```

x = torch.tensor([1, 2])
y = x.add(10) ----- torch.add 적용
print(y)
print(x is y)
print('-----')
y = x.add_(10) ----- torch.add_ 적용
print(y)
print(x is y)

```

다음과 같이 torch.add와 torch.add_의 결과는 같지만, x가 y와 같은지(x is y) 물었을 때는 다른 값을 출력함.

```

tensor([11, 12])
False
-----
tensor([11, 12])
True

```

즉, torch.add의 결과는 새로운 메모리 공간이 할당되어 저장되기 때문에 x와 y가 같은지 물었을 때 False라는 결과를 출력한 반면, torch.add_는 새로운 공간 할당 없이 기존의 메모리에 위치한 값을 대체함. 참고로 메서드에 _ 표시가 있다면(예 torch.clamp_()) 기존의 메모리 공간에 있는 값을 새로운 값으로 대체하겠다는 의미로 받아들이면 됨.

모델이 정확하게 예측한 이미지만 출력하기 위한 함수를 정의함. 이때 출력 방법에 대한 정의도 함께 진행해 보겠음.

```
def plot_most_correct(correct, classes, n_images, normalize=True):
    rows = int(np.sqrt(n_images)) ----- np.sqrt는 제곱근을 계산(0.5를 거듭제곱)
    cols = int(np.sqrt(n_images))
    fig = plt.figure(figsize=(25,20))
    for i in range(rows*cols):
        ax = fig.add_subplot(rows, cols, i+1) ----- 출력하려는 그래프 개수만큼 subplot을 만듭니다.
        image, true_label, probs = correct[i]
        image = image.permute(1, 2, 0) ----- ①
        true_prob = probs[true_label]
        correct_prob, correct_label = torch.max(probs, dim=0)
        true_class = classes[true_label]
        correct_class = classes[correct_label]

        if normalize: ----- 본래 이미지대로 출력하기 위해 normalize_image 함수 호출
            image = normalize_image(image)

        ax.imshow(image.cpu().numpy())
        ax.set_title(f'true label: {true_class} ({true_prob:.3f})\n' \
                    f'pred label: {correct_class} ({correct_prob:.3f})')
        ax.axis('off')

    fig.subplots_adjust(hspace=0.4)
```

① image.permute는 축을 변경할 때 사용함. 축을 변경한다는 의미를 예제로 확인해 봄.

```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(x)
x.permute(1, 0) ----- 차원(축) 0과 1을 바꿉니다.
```

다음은 0과 1의 차원(축)이 변경된 결과임. 즉, 행과 열이 변경되었음.

```
tensor([[1, 2, 3],
        [4, 5, 6]])
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

모델이 정확하게 예측한 이미지를 출력하기 위해 `plot_most_correct()` 함수를 호출함. 호출할 때 모델이 정확하게 예측한 이미지(`correct_examples`)에 대해 출력하도록 함.

코드 6-64 예측 결과 이미지 출력

```
classes = test_dataset.classes
N_IMAGES = 5
plot_most_correct(correct_examples, classes, N_IMAGES)
```

다음 그림은 모델이 정확하게 예측한 이미지에 대한 출력 결과임.

▼ 그림 6-22 테스트 데이터셋에 대한 VGG 모델의 예측 결과



결과의 정확도가 높지 않음. 역시 동일하게 데이터셋을 늘려 준다면 정확도에 대한 성능이 높아질 것임.

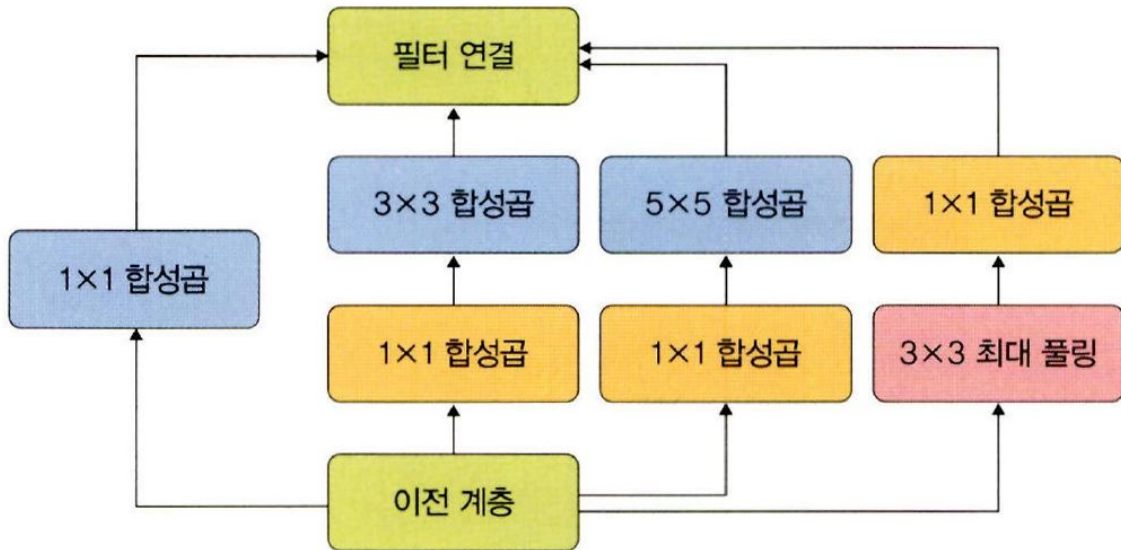
6.1.4 GoogLeNet

GoogLeNet은 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망임.

깊고 넓은 신경망을 위해 GoogLeNet은 인셉션(inception) 모듈을 추가했음. 인셉션 모듈에서는 특징을 효율적으로 추출하기 위해 1×1, 3×3, 5×5의 합성곱 연산을 각각 수행함. 3×3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 풀링 연산에서는 드물게 패딩을 추가해야 함. 결과적으로 GoogLeNet에 적용된 해결 방법은 희소 연결(sparse connectivity)임.

CNN은 합성곱, 풀링, 완전연결층들이 서로 밀집(dense)(정교하고 뽁뽁하게)하게 연결되어 있음. 뽁뽁하게 연결된 신경망 대신 관련성(correlation)이 높은 노드끼리만 연결하는 방법을 희소 연결이라고 함. 이것으로 연산량이 적어지며 과적합도 해결할 수 있음.

▼ 그림 6-23 GoogLeNet의 인셉션 모듈



인셉션 모듈의 네 가지 연산은 다음과 같음.

- 1x1 합성곱
- 1x1 합성곱 + 3x3 합성곱
- 1x1 합성곱 + 5x5 합성곱
- 3x3 최대 풀링(maxpooling) + 1x1 합성곱(convolutional)

딥러닝을 이용하여 ImageNet과 같은 대회에 참여하거나 서비스를 제공하려면 대용량 데이터를 학습해야 함. 심층 신경망의 아키텍처에서 계층이 넓고(뉴런이 많고) 깊으면(계층이 많으면) 인식률은 좋아지지만, 과적합이나 기울기 소멸 문제(vanishing gradient problem)를 비롯한 학습 시간 지연과 연산 속도 등의 문제가 있음. 특히 합성곱 신경망에서 이러한 문제들이 자주 나타나는데, GoogLeNet(혹은 인셉션이라고도 불림)으로 이러한 문제를 해결할 수 있다고 생각하면 됨.

6.1.5 ResNet

ResNet은 마이크로소프트에서 개발한 알고리즘으로 “Deep Residual Learning for Image Recognition”이라는 논문에서 발표되었음. ResNet 핵심은 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 레지듀얼(residual) 개념을 고안한 것임.